Check for updates

Deep Generative Models in Engineering Design: A Review

Lyle Regenwetter
Department of Mechanical Engineering,
Massachusetts Institute of Technology,
Cambridge, MA 02139
e-mail: regenwet@mit.edu

Amin Heyrani Nobari
Department of Mechanical Engineering,
Massachusetts Institute of Technology,
Cambridge, MA 02139
e-mail: ahnobari@mit.edu

Faez Ahmed
Department of Mechanical Engineering,
Massachusetts Institute of Technology,
Cambridge, MA 02139
e-mail: faez@mit.edu

Automated design synthesis has the potential to revolutionize the modern engineering design process and improve access to highly optimized and customized products across countless industries. Successfully adapting generative machine learning to design engineering may enable such automated design synthesis and is a research subject of great importance. We present a review and analysis of deep generative machine learning models in engineering design. Deep generative models (DGMs) typically leverage deep networks to learn from an input dataset and synthesize new designs. Recently, DGMs such as feedforward neural networks (NNs), generative adversarial networks (GANs), variational autoencoders (VAEs), and certain deep reinforcement learning (DRL) frameworks have shown promising results in design applications like structural optimization, materials design, and shape synthesis. The prevalence of DGMs in engineering design has skyrocketed since 2016. Anticipating the continued growth, we conduct a review of recent advances to benefit researchers interested in DGMs for design. We structure our review as an exposition of the algorithms, datasets, representation methods, and applications commonly used in the current literature. In particular, we discuss key works that have introduced new techniques and methods in DGMs, successfully applied DGMs to a design-related domain, or directly supported the development of DGMs through datasets or auxiliary methods. We further identify key challenges and limitations currently seen in DGMs across design fields, such as design creativity, handling constraints and objectives, and modeling both form and functional performance simultaneously. In our discussion, we identify possible solution pathways as key areas on which to target the future work.
[DOI: 10.1115/1.4053859]

Keywords: artificial intelligence, data-driven design, design automation, design representation, generative design, machine learning, product design

1 Introduction

The human design process is a ubiquitous element of modern society, playing a critical role in the technologies producing the food we eat, the products we use, and the spaces in which we live. Accelerating the design process through automation can reduce cost and increase industrial productivity, which would be immensely desirable for global productivity and prosperity. Integrating AI into the design process can alleviate dependence on human experts and revolutionize user customizability, providing specialized products for individual users without the prohibitive cost of manual design. Driven by the widespread potential to advance global equity and prosperity through design automation, methods such as "generative design" have recently emerged alongside advanced computing and automation technologies.

"Generative design" is the process in which algorithms directly synthesize designs either via explicit programming or implicit learning. Early generative design methods leaned heavily on explicit programming of human design expertise through manually defined design representation methods like grammars [1]. While practical for explicitly encoding design constraints and objectives, these rule-based frameworks ignored opportunities for implicit leaning on information and knowledge encoded in the vast expanse of existing designs. As the availability of computational resources increased over the past decade, data-intensive methods like deep learning opened doors to successfully automate complex human tasks such as image processing and natural language processing.

In deep learning, data are propagated through sequential layers to learn progressively higher-level meaning, an architecture generally known as an artificial neural network or just neural network (NN) [2]. Most of the deep learning-based approaches pioneered during the 2010s leveraged extensive quantities of data to avoid explicit feature engineering. This trend is mirrored in engineering design with algorithms learning data distributions instead of requiring them to be predefined. Among these algorithms are deep generative models (DGMs)—deep learning models that can approximate complicated, high-dimensional probability distributions using a large dataset. In this review article, we specifically define "deep generative models" as algorithms that are capable of generating new samples using deep learning. Generative adversarial networks (GANs) and variational autoencoders (VAEs) are two classes of DGMs that have demonstrated compelling synthesis of images, text, and tabular data in numerous domains. Considering that images, text, and tabular data are all common representation methods for design, one might assume that DGMs should be capable of synthesizing full designs as well with relative ease. However, several unique properties of the generative design task pose particular challenges for DGMs. Many of these challenges are so fundamental that the future success of DGMs in engineering design is largely contingent on the ability to overcome them. We list four of these challenges below:

(1) Modeling design performance: Real-world functional performance is critical in many engineering design tasks. Developing performance-aware DGMs capable of synthesizing designs for a given set of target requirements (a process termed as inverse design) is a challenging task that is exacerbated by the computational cost of numerical simulation and the even greater difficulty of real-world evaluation.
(2) Data sparsity: Compared to other research fields like computer vision, which have massive publicly available datasets, the availability of large, well-annotated, public datasets in engineering is severely lacking. Furthermore, even when

Contributed by the Design Automation Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received October 20, 2021; final manuscript received January 29, 2022; published online March 18, 2022. Assoc. Editor: Jitesh H. Panchal.

 Copyright © 2022 by ASME

om http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2hAAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UiBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

data are available, the distribution of the data often do not cover the design space evenly, with much sparsity often observed in the data.

(3) The creativity gap: In conventional DGM applications, the overarching goal is to mimic the training data and emulate existing designs. In engineering design, emulation of existing products is often undesirable. Designers typically aim to introduce products with novel features to target new market segments.

(4) Usability and feasibility: For synthesized designs to be physically fabricated, they must be physically feasible. Furthermore, designs must be encoded in a data representation that contains enough parametric detail to be converted into a representation usable for fabrication.

Over the past few years, the design community has made substantial progress in using generative machine learning (ML) models to create new designs. DGMs have been applied to a broad range of design tasks such as structural optimization, materials design, and shape synthesis. Over time, researchers have introduced increasingly advanced methods, which have begun to address some of the aforementioned challenges. For example, many works have proposed approaches to incorporate design performance and optimization into DGM training. Other works have explored incorporating novelty and creativity into DGMs. Despite these advancements, DGMs for engineering design are still in their infancy and will require further efforts to effectively overcome these fundamental challenges.

Our primary goal in this study is to help build cohesion between the countless active researchers in the design field working with DGMs and furthermore provide a starting point for researchers entering the field. In particular, we seek to provide researchers with a reference guide in planning projects in the data-driven generative design space. To this end, we provide an overview of common methods and tools (Sec. 2), a discussion of different data parameterization methods (Sec. 3), a review of potentially relevant research across various design domains (Sec. 5), an overview of relevant datasets (Sec. 6), and an analysis of common challenges in the field (Sec. 7). Figure 1 provides an overview of the standard process to apply DGMs in the engineering design.

2 Overview of Deep Generative Models

Deep generative machine learning approaches share the goal of high-quality synthesis but significantly vary in methodology. In practice, we identify four common approaches to generate designs: direct generation using deep neural networks (DNNs), adversarial generation with GANs, generation from embedding vectors using VAEs, and sequential generation using reinforcement learning (RL). In the design community, we observe that GANs, VAEs, and RL are most commonly used for design synthesis. While DNNs as well as extensions like recurrent neural networks (RNNs) are occasionally used for direct design synthesis, they are more frequently used for nongenerative tasks. In this section, we briefly discuss the background and methodology of GANs, VAEs, and RL.

2.1 Generative Adversarial Networks. Originally introduced in 2014, GANs [3] found initial success with convincing image synthesis performance [4–7]. We provide an introduction of GANs but refer the reader to Ref. [8] for a detailed overview.

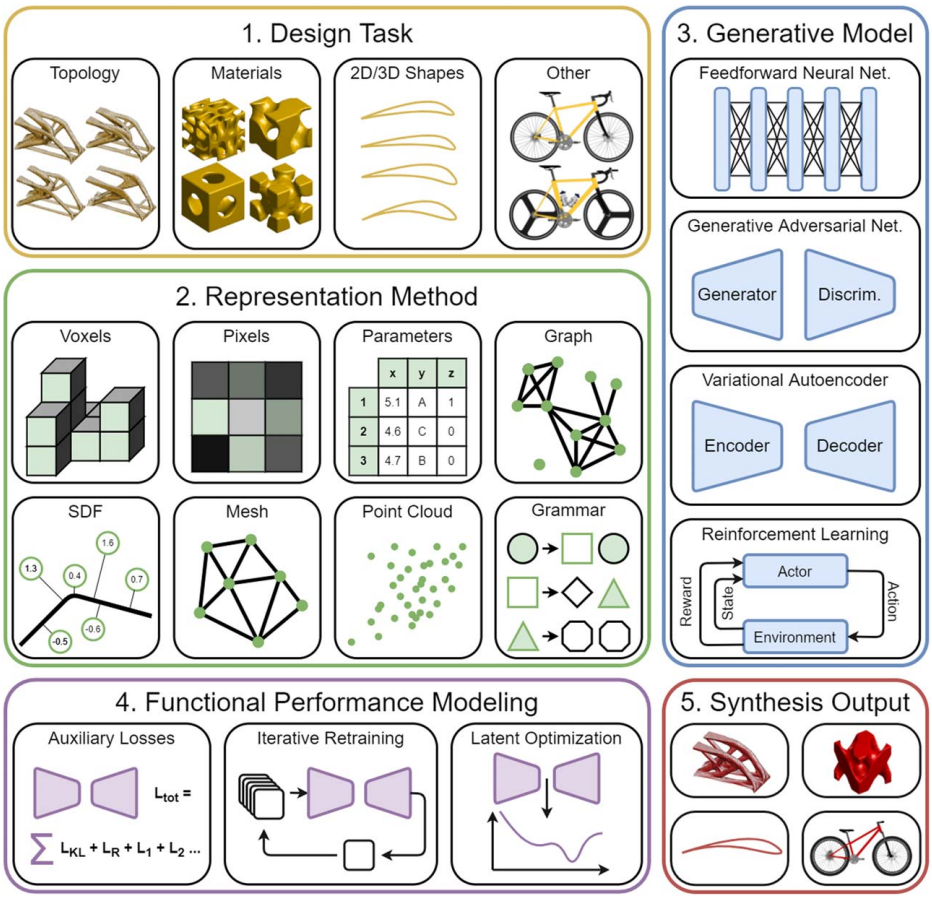


Fig. 1 This figure outlines the typical components of design synthesis problems using deep generative models. Some design problems are more suitable for specific design representation methods, which also influence the type of deep generative model architectures required.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UiBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

A generative adversarial network [3] consists of two models—a *generator* and a *discriminator*. The generator G maps an arbitrary noise distribution to the data distribution, in our case the distribution of designs, and can thus generate new data; simultaneously, the discriminator D learns to distinguish between real and generated data. Both models are usually built with deep neural networks. As D improves, G also improves as it learns to generate data that fools D.

2.1.1 Common Challenges in Training Generative Adversarial Network Models. GANs are often considered difficult to train, suffering from training instability stemming from several sources [9,10]. One common issue in GAN training occurs when the discriminator overpowers the generator, easily distinguishing generated samples and causing the gradient to the generator to vanish, effectively halting the generator's training. This issue has been addressed by many researchers. For example, the WGAN replaces the discriminator with a critic, modifies the GAN's loss function to estimate the Wasserstein (Earth Mover's) distance between the original data and generated data distributions, and modifies the training process [11,12].

Another problem that GANs face is the issue of "mode collapse," where the generator fails to encompass all modes in the data distribution or even generates only a handful of unique samples that are capable of fooling the discriminator. To overcome these issues, researchers have developed novel algorithmic techniques [13–15] to reward diversity in samples.

2.1.2 Generative Adversarial Network Conditioning. In the design domain, we often have design constraints, requirements, and objectives that any generated design should satisfy. To use DGMs for such problems, these requirements should be imposed on them. For example, we may seek to train a DGM to generate bikes, but depending on our user, we may want to constrain it to generate only roadbikes or mountain bikes without retraining for each generation task. Model conditioning is one method to do this. Several proposed approaches add conditioning to the GAN using a condition vector, which is intended to be interpretable. Typically GANs are discretely conditioned by feeding the condition vector into both the generator and discriminator, in a configuration known as a conditional GAN (cGAN) [16]. Instead of feeding the condition vector into the discriminator, an auxiliary network and cross entropy loss can instead be used to reconstruct the condition vector from the generated samples in a configuration known as an information maximizing GAN (InfoGAN) [17]. Conditioning is also essential in design applications where inverse design is being done on performance metrics, which often exist in continuous spaces (e.g., stiffness, lift coefficient, drag coefficient, density). Researchers have come up with continuous conditioning solutions for GANs such as the regressional GAN [18], continuous conditional GAN (CcGAN) [19], and performance conditioned diverse GAN [20].

2.2 Variational Autoencoders. Introduced in 2013, variational autoencoders found significant success in many machine learning applications. Autoencoders are unsupervised embedding algorithms consisting of an *encoder* that maps an input design into a (typically) lower-dimensional latent space and a *decoder* that reconstructs the design as accurately as possible from the latent space. The encoder and the decoder are conventionally implemented using deep neural networks. To generate new samples, latent vectors are sampled from the latent space and fed through the decoder. Typically, the distribution of the real data mapped to the latent space of an autoencoder is sparse, meaning that sampling a realistic latent vector is difficult. This limitation is addressed with the introduction of the variational autoencoder (VAE), first proposed by Kingma and Welling [21]. The variational autoencoder adds in a probabilistic sampling in the latent space that regularizes the latent distribution. In practice, the VAE's encoder outputs n means and n variances, from which n-dimensional latent vectors are sampled before decoding. To maintain a predictable latent space distribution, the VAE adds a Kullback–Liebler divergence [22] loss between the distribution of the latent space and a standard Gaussian. Interested readers are encouraged to refer to the literature [23] for a more detailed overview.

2.2.1 Conditional Variational Autoencoders. Just as we do for GANs, we may also seek to condition VAE training on design constraints or user preferences. The VAE has a natural advantage over the GAN in that its latent space is typically already structured. Since this structure may be fairly weak and difficult to interpret, explicitly conditioning VAEs may still be desirable. The conditional VAE (cVAE) [24] extends on the conventional VAE by adding a conditioning vector as an input to both the encoder and the decoder and helps achieve this goal.

2.3 Reinforcement Learning. Reinforcement learning fundamentally differs from the other DGMs discussed in that it learns without a dataset in an unsupervised fashion through a large set of trial-and-error interactions between an *actor* and an *environment* [25]. This is typically done through some reward signal being sent to the actor after taking actions, based on the effects of said actions on the environment. In this scenario, the actor's goal is to maximize the rewards it receives by making decisions (i.e., taking actions) such that the total reward is maximized. From this point of view, reinforcement learning can be thought of as an approach similar to optimization, where an objective (maximizing the reward) is being optimized.

One of the first attempts at introducing deep learning to the reinforcement learning approach was done in 2013 by Mnih et al., when they introduced deep learning to a reinforcement learning process known as Q-learning [26]. Q-learning refers to learning the state-action value function or Q-function, which is a progressively updated estimate of the expected reward to be received from taking a particular action in a particular state. Mnih et al. attempted to learn the Q-function using convolutional neural networks (CNNs). Many deep RL techniques have been introduced since this first work by Mnih et al. Further exploration of the details of these approaches is left to the reader.

In practice, when applying RL to design applications, the design process is usually broken down into a sequential process of building a design or altering existing designs in steps (i.e., actions taken to alter or expand the current state of a design being generated) and the reward is measured by the quality or performance of the resulting design (i.e., the environment). While RL requires no dataset, this advantage is balanced by dependence on meaningful and reliable reward signals, which may often require a high-fidelity simulation environment. One major benefit of RL over GANs and VAEs is the fact that the reward function can be set based on any objective, which does not need to be differentiable. In contrast, any objective added to the loss function of a GAN or VAE must be differentiable since GANs and VAEs are trained using the gradient-based optimization [15].

3 Overview of Design Representation Methods

In this section, we discuss common design representation methods seen in DGMs for engineering design, which are visualized in Sec. 2 of Fig. 1. We include a definition and discuss the pros and cons of each method.

3.1 Images. Design data often come in the form of images (e.g., microstructure scans) or can be represented in an image form (e.g., topology optimization). An image consists of a rectangular grid of pixels, each of which contains a color parameter. They are commonly represented by third-order tensors ($height \times width \times channels$). Common color schemes are black-and-white (boolean color channel), grayscale (integer color channel), and color (3–4 integer color channels). *Pros:* The image is an information-rich representation and can capture many details of a design. The use of

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8rtDy_sQpyXO by Universita Di Pisa user on 19 Novemb

convolution/convolution-transpose filters in deep learning provides a convenient tool for learning/generation of both high-level and low-level features as well as upsampling/downsampling. Many cutting edge ML techniques are pioneered in the computer vision domain and are often directly applicable to images. *Cons:* Representing designs using pixels means that the generated design images can be infeasible for downstream tasks. Accurately fabricating designs based on images can be difficult or impossible. Even performance evaluation using conventional simulation tools like FEA or computational fluid dynamics (CFD) can require an intermediate conversion from an image to a 3D model. The poor usability of images is exacerbated by the prevalence of artifacts in many applications (hanging pixels, disconnected geometry, etc.). Artifacts are especially common when training on (typically) small datasets in the design domain since training is often terminated early due to overfitting concerns. All in all, images can be considered surrogate representations of engineering designs and may lack domain knowledge and information on the physical realization of the design. Therefore, DGMs using images as representations often have a gap between the generated images and the actual design they are representing.

3.2 Voxelizations. Voxels are 3D grid points that are effectively the 3D equivalent of pixels. As such, voxelizations share many characteristics with images. In practice, voxels are not conducive to "color" parameterization and are typically represented as booleans (space versus object). This effectively makes them third-order tensors with dimension (height × width × depth). *Pros:* Voxelizations support 3D convolution, which can learn high-level and low-level features in 3D. *Cons:* Compared to images and other representations, the curse of dimensionality is especially pronounced with voxels, with the number of parameters scaling with the cube of spatial resolution. Voxelizations share the same issues as images. Their usability is limited in downstream tasks, and artifacts are very prevalent. Like images, voxelizations serve as surrogate representations (often representing CAD models, which originate from parametric representations or 3D shapes, which originate from meshes). Like many other representations such as point clouds and signed distance fields, voxelizations often require conversion before they can be used in downstream tasks. For example, they are often converted to boundary representation (BRep) or polygonal representations, which are often the native parameterizations of rendering and graphics software, finite element analysis, and computational fluid dynamics simulation.

3.3 Point Clouds. Point clouds are simple collections of points, often in 3D space, which are defined to be within some object. *Pros:* Point clouds can represent arbitrarily complex geometry with a finite number of points, although fidelity may vary. Point clouds are often the native output of 3D scanning software, making them relatively easy to create [27]. *Cons:* Like voxelizations and signed distance functions, Point clouds often require conversion to BRep or polygonal representations such as meshes [28] for downstream tasks.

3.4 Meshes. Meshes are a common method to represent objects in 3D space. Triangular meshes are by far the most commonly used form. Triangular meshes are the native representation used in many computer graphics algorithms and software, as well as many finite element tools. *Pros:* Meshes can be directly visualized and simulated in many FEA or CFD tools, enabling easy pipelines for performance evaluation using numerical methods. A mesh can be considered a specialized type of graph and can leverage graph operators like graph convolutional operators. *Cons:* In contrast to other representations like voxelizations and point clouds, meshes are more challenging to directly generate using machine learning methods despite recent advances in algorithms that directly generate meshes [29–31].

3.5 Signed Distance Functions. The signed distance function/field (SDF) is a representation method that consists of a (typically 3D) functional map from a coordinate point to an SDF value. The magnitude of this value indicates the distance to the nearest point on the surface of the object and the sign indicates whether the point is inside or outside the object. SDFs themselves can be represented in many ways, for example, as a rasterized grid in which each "voxel" contains a continuous numerical value denoting the SDF value at that point. *Pros:* SDFs can serve as a convenient intermediate parameterization for many learning tasks. *Cons:* Like point clouds or voxels, SDFs are difficult to use in downstream tasks without first converting to BRep or polygonal representations.

3.6 Parameterizations. We use the term "parametric" data to encompass any design representation consisting of a collection of design parameters where any spatial or temporal significance of parameters is unknown. Most parametric data can be organized in a tabular form with each row being a collection of parameters representing a single design and each column describing a design parameter. Tabular design data often consist of a collection of mixed-datatype parameters where relations between these parameters may be unclear or nonexistent. Since parametric data may come in many varieties, the pros and cons discussed may not apply to every case. *Pros:* Quality parametric data are typically very information dense (i.e., requiring fewer parameters to encode the same level of geometric detail). Although spatially organized representations such as pixels or voxels encode designs with uniform information density, parametric data can contain more detail in design-critical areas without the need for upsampling the entire representation. This information density often comes with a lower dimensionality, which can make optimization of parametrically represented designs significantly easier. Parametric data may also be more supportive of downstream tasks, especially if design parameters are human interpretable. For example, a detailed enough design parameterization may allow generated designs to be directly fabricated using conventional (nonadditive) manufacturing techniques. Human-interpretable parameterizations can also give human designers a tractable method to interact with generative methods to allow for human-in-loop design. Finally, design parameters can sometimes be directly linked to the latent space of a generative method, as demonstrated in numerous works [32,33], creating a pipeline to directly condition design generation on high-level design goals. Linking design parameters with latent variables has several potential advantages, such as enabling more effective optimization or inverse design using generative methods. *Cons:* Learning parametric data can be particularly challenging. Parametric data commonly use mixed datatypes and inherits the training challenges of the constituent components. Multimodal distributions, skewed categories, non-Gaussian distributions, data sparsity, and poor data scaling additionally make the application of DGMs and training very difficult. Since methods that are robust to all of the mentioned challenges are difficult to come by, successfully applying existing methods to the parametric data domain can be hard. Finally, since parametric data may be nontrivial to convert to 3D models, generated parametric designs may be challenging to evaluate using numerical simulations or through qualitative visualization.

3.7 Grammars. Grammars are representation methods consisting of variables, terminal symbols, nonterminal symbols, and a set of rules. Rules describe how nonterminal symbols can expand into other terminal and nonterminal symbols. The most prevalent grammars in engineering design are graph and spatial grammars [1], and they have been applied in a wide variety of applications, such as the design of satellites and electromechanical systems. Grammars can be especially useful to dictate feasible assembly hierarchies of design components as in Ref. [34]. *Pros:* Grammars can explicitly constrain design spaces to feasible or desirable regions by nature of their construction, thereby encoding domain knowledge. *Cons:* Grammars are challenging to implicitly learn and often

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2hAAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

must be manually defined. Grammars can also restrict the exploration of the design space. A survey on grammar-based design synthesis approaches is provided in Ref. [1].

3.8 Graphs. Graphs are a highly flexible representation method consisting of nodes and edges, which can be directed or undirected. Graph-based representations have proven successful for many different aspects of design generation and optimization and provided avenues for describing complex systems efficiently. *Pros:* Graphs are highly adaptable and are capable of representing many different kinds of complex systems and designs [35–38]. Graphs also provide representations for design processes and modeling complex interactions in systems [39,40], which may enable methods for automating the design of systems or modeling interpart dependencies. Graph neural networks (GNNs) [41–45] provide an excellent tool for machine learning on graphs. *Cons:* Despite the developments of GNNs in the computer science community [46–49] and their success in molecular graph generation [50,51], there is less usage of graph-based DGMs in the design community, possibly due to the lack of graph-based design datasets.

4 Literature Review Methodology

Section 5 discusses specific works that apply deep generative models to engineering design or make advancements to existing generative ML methods in the context of engineering design. We consider works based on a predefined scope, with each work we discuss meeting the following selection criteria. Note that these criteria only apply to the engineering design papers presented and that the works we cite to add context to or substantiate the discussion of fundamentals, applications, and datasets need not adhere to these rules.

(1) We limit our consideration specifically to papers involving deep generative models, focusing on variational autoencoders, generative adversarial networks, and reinforcement learning in particular. Works must utilize deep learning. Works only considering design optimization are excluded.
(2) We only consider work specifically relevant to engineering design and not other domains (such as computer science).
(3) We consider only work published between Jan. 2014 and Sep. 2021, when we conclude our review, as many pivotal works in deep learning (CNNs, VAEs, and GANs) were introduced in this period.

To identify works, we specifically searched for "generative adversarial network," "variational autoencoder," and "reinforcement learning" in Google Scholar. We initially confined our search to a set of known design venues, specifically the *Journal of Mechanical Design*, the *Proceedings of the International Design Engineering Technical Conferences*, *Computer-Aided Design Journal*, *International Conference on Engineering Design, and Artificial Intelligence for Engineering Design, Analysis, and Manufacturing Journal*. This helped us identify an initial set of seed papers. From all search results identified through these methods, 41 papers were deemed to be relevant and included. The relevance was decided by independent assessment by two raters, who are also authors of this article. In papers where there was a disagreement, all authors discussed them and mutually decided on their classification. Next, papers cited in these seed papers were considered and added to this paper. Twenty-two papers from other venues were deemed to be relevant and included as well. Of the 63 papers, 48 were published between Jan. 2019 and Aug. 2021, while a mere 15 were published between Jan. 2016 and Dec. 2018, indicating strong growth in the field in recent years.

5 Application Domains in Engineering Design

Engineering design encompasses a wide variety of applications, ranging from designing aircraft models to small-scale metamaterials. To structure different types of applications using DGMs, we grouped them into a few categories of application areas, which are discussed in this section and are reported in Table 1.

5.1 Deep Generative Models in Topology Optimization. Topology optimization (TO) is a research field with a long history of research and methods (Fig. 2). TO searches a design space to find an ideal spatial distribution of material to optimize some predefined objective. Common areas of application include solid mechanics [111,112], fluid dynamics [113,114], additive manufacturing [115,116], and heat transfer [117,118]. While topology optimization is considered a method for generative design, standard TO does not leverage deep learning and is not considered a DGM. In recent years, however, several papers have proposed methods to use TO and DGMs together, in many cases to address the computational cost of TO on large amounts of data. Despite differences between architectures, we found that many DGMs for TO share certain characteristics. In particular, due to the predominant use of

Table 1 Characterization of application studies by domain and architecture

	Topology optimization	Materials	2D shape synthesis	3D shape synthesis	Other domains
Plain NN	[52]^{i} [53]^{vi} [54]^{pa} [55]^{i}	[56]^{p} [57]^{i} [58]^{i}			
GAN	[59]^{i} [60]^{i} [61]^{i} [62]^{i}	[63] [64]^{i} [65]^{vi} [66]^{v} [67]^{i} [68]^{vi}		[69]^{c}	
GAN + conditioning	[70]^{i} [71]^{i} [72]^{i} [73]^{i}		[74]^{p} [75]^{p} [76]^{p} [33]^{ph}		
New GAN based	[72]^{i}		[15]^{p}	[77]^{v}	[78]^{i}
AE/VAE	[79]^{i}	[80]^{i} [81]^{i} [82]^{i} [32]^{i} [83]^{i}		[84]^{v}	[85]^{p}, [86]^{p}, [87]^{p}
cVAE		[88]^{b} [89]^{i}			[85]^{p}[90]^{i}
New VAE based		[91]^{i}		[92]^{s}	[93]^{ip}
RL based			[94]^{p} [95]^{p}		[96]^{ip} [34]^{g} [97]^{ip} [98]^{p} [99]^{p}
Other method	[100]^{v} [101]^{i}	[102]^{i} [103]^{i} [67]^{i} [104]^{i} [105]^{i} [106]^{i}	[95]^{p}		[100]^{v} [107]^{p} [34]^{g} [108]^{ip} [109]^{ip} [110]^{g}
+Style transfer	[79]^{i}	[64]^{i} [65]^{vi} [80]^{i} [57]^{i} [104]^{i}			
+Genetic Alg.		[82]^{i} [105]^{i}		[92]^{s}	
+Bayesian Opt.		[64]^{i} [83]^{i}	[76]^{p}		

Note: Some works use multiple architectures or fit into multiple application domains and are listed more than once. We additionally classify works by datatype used: ^{i}Image, ^{c}Point cloud, ^{v}Voxelelization, ^{m}Mesh, ^{s}Signed distance field, ^{g}Grammar, ^{h}Graph, and ^{p}Parametric (other parameterization).
[a]Model-based reconstruction using parameters in a lower-dimensional space.
[b]Technically not images, but use an image-like parameterization structure.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2hAAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb



Fig. 2 Sample topologies generated by 3D topology optimization

voxelized or pixelized representations, many hybrid TO-generative ML models use methods originally developed for computer vision applications, such as convolutional neural networks and super-resolution [53,59].

5.1.1 Supervised Generation of Optimized Topologies. To avoid the computational cost of topology optimization, DGMs have been used to predict final optimized topologies directly. This approach can also be used as an initialization technique for conventional TO, which allows conventional TO algorithms to rapidly converge, saving computational cost. We consider a baseline for DGMs in TO, in which an existing generative architecture is applied to a dataset of TO-generated designs and the network is trained to generate samples mimicking the training data. A few papers fall within this category of methods: Rawat and Shen [60] train a WGAN on a dataset generated through TO and train an auxiliary network to additionally predict performance metrics. Sharpe and Seepersad [70] expand on this baseline with a cGAN conditioned on volume fraction and load location. Guo et al. [79] train a VAE with an additional style transfer [119,120] loss on a TO-generated dataset for heat transfer and propose iterative strategies for targeted design optimization using the VAE's latent space. Style transfer is discussed further in Sec. 5.2.

5.1.2 Iterative Deep Generative Models Training, Filtering, and Human Guidance in Topology Optimization. Oh et al. [61,62] propose a method that expands on the baseline generative-network fitting by iteratively synthesizing new designs, optimizing them in TO, and then dropping designs that are too similar from the full collection of designs. The authors use a modified boundary equilibrium GAN [121], which extends on the WGAN and begin training on a collection of existing TO-generated topologies. However, unlike the previously discussed generative-network-fitting approaches used in the literature [60,70,79], the retraining and re-optimization allows the framework to generate, optimize, and explore new areas of the design space. The proposed method is applied to the problem of wheel design, with the key motivation being to find a tradeoff between the aesthetics of real designs and the structural performance of designs found through TO. The authors present strong empirical results on this wheel design problem.

The issue of gaps in the design space of topologies is also addressed by Fujita et al. [101], who propose an approach leveraging a variational deep embedding (VaDE) [122]. An initial dataset is generated using TO. The proposed method sequentially identifies voids in the design space using the VaDE, decodes designs from the void, optimizes them using TO, and then adds them to the training dataset.

Although the aforementioned works have proposed methods with automated retraining steps, human input can also be injected into the training process. For example, Valdez et al. [73] proposed a cGAN-based human-in-loop topology design generation framework in which the designer iteratively selects design clusters to gradually hone in on preferable designs.

5.1.3 Deep Generative Models for Topology Optimization That Utilize Super-Resolution. Since one of the key limitations of topology optimization is computational cost, which scales with the resolution, a major focus in DGMs for TO has focused on attaining high-resolution TO-like results. A common approach uses super-resolution, which is a technique used in computer vision to convert low-resolution images to high-resolution ones. Several works expand on the baseline DGM-in-TO framework by learning from a low-resolution TO-generated dataset, then performing super-resolution on synthesized topologies, such as Yu et al. [72], who add a cGAN for upscaling. A similar approach proposed by Li et al. [59] uses a super-resolution GAN [123] to generate high-resolution optimal topologies for heat transfer problems after generating low-dimensional topologies on a different GAN. Other researchers have proposed transfer learning for the super-resolution task instead of using GANs or VAEs for super-resolution. For example, Behzadi and Ilieş [53] train a feedforward NN-based model to predict optimized topologies directly without any discriminator and using the MSE loss. Once they have trained their model on the low-resolution data, they lock the model weights and add a few layers to the model to increase the output resolution and only train said layers to obtain high-resolution samples. This avoids the need for a large quantity of high-resolution data or the extra time required to train a high-resolution model from scratch. Many papers that have applied super-resolution techniques demonstrate that after initial training, their framework consistently generates near-optimal topologies many times faster than classic TO [59,72].

5.1.4 Improving Deep Generative Model-Based Topology Generation Using Physical Properties. Several papers have succeeded in improving the baseline performance of DGMs for topology generation through the use of physical properties of the design domain. For example, Nie et al. [71] proposed to adapt the cGAN architecture Pix2Pix [124] to generate synthetic topologies based on spatial fields of various physical parameters (displacement, strain energy density, and Von Mises stress) as input to the generator. Topologies generated by TO are taken as ground truth for training. The authors also propose a new generator architecture combining the Squeeze and Excitation ResNet [125,126] with U-Net [127]. Cang et al. [55] used a neural network to generate optimized topologies from loading conditions. They propose an approach to rapidly evaluate the deviation of proposed solutions from the problem's optimality conditions. They then progressively augment their dataset during training by recalculating optimal solutions (using TO) for proposed solutions that are in greatest violation of these conditions.

5.1.5 Other Deep Generative Model Approaches in Topology Generation. Sosnovik and Oseledets [52] proposed to use gradient information from a topology distribution to inform estimates of a final topology through a DNN. Their DNN takes both the density distribution and the gradient of this density distribution from some intermediate step of a TO process to generate an estimate of the final topology without waiting for TO convergence. The authors demonstrate binary accuracy of 98% when predicting the final topology after only five iterations of TO, when the full TO process would take 100 iterations to complete.

In a different kind of approach, Keshavarzzadeh et al. [54] addressed the generalization problem of training for different scales and different domains. They parametrically represent shapes in both 2D and 3D using their proposed disjunctive normal shape model (DNSM) [54]. By using this DNSM, they create a platform for resolution-independent shape reconstruction. They then train an NN model to generate optimal topologies given boundary conditions and other problem-specific information

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

in the DNSM space, which then can be reconstructed at any resolution using the DNSM. They demonstrate that their DNSM method overcomes the super-resolution problem across a variety of problems and domains.

5.2 Microstructure, Nanostructure, and Metamaterials. Many design applications not only require the need to design the topology or shape of the artifact but also require the material properties within it (Fig. 3). Inverse design of materials is one of the key elements of computational materials science. The most common approach to developing inverse materials design frameworks is the development of process–structure–property (PSP) links, i.e., understanding how a particular material processing approach impacts its microstructure and how the corresponding microstructure impacts its physical properties [128]. Designing material microstructures for direct use is difficult, as there must then be some fabrication process to generate a material with the target microstructure. A common goal in computational materials science is microstructure image "reconstruction," and in other words, generating microstructure images that exhibit certain characteristics. Bostanabad et al. [128] give an overview of the microstructure characterization and reconstruction field. Reconstruction can accelerate downstream tasks like augmenting the training data of networks that attempt to model PSP links. Better modeling PSP links can in turn create more accurate generative material design pipelines. Many classes of DGMs have been applied to this reconstruction task, including GANs [64], VAEs [80], and convolutional deep belief networks [102,103,106,129]. Other studies attempt to bridge the gap between microstructure and properties in generative tasks using trained black-box surrogates, such as Tan et al.'s work [63]. Most research studies focus on 2D microstructure images, although several also consider 3D voxelizations [66,68].

Since reconstruction often involves mimicking existing microstructures, several papers have applied style transfer [119,120] to the problem as part of a DGM architecture. Style transfer is in essence a loss between two images that attempts to capture the difference in "style" between the images. It is typically calculated by comparing the intermediate layers of an auxiliary style convolutional neural network. Cang et al. [80], for example, proposed a VAE with style transfer, which is targeted at applications where only a small set of training data is available. Li et al. [57] also utilized style transfer in their proposed transfer learning reconstruction framework. In another approach using style transfer, this time using a vanilla GAN, Yang et al. [64] expanded on the reconstruction task by attempting inverse design on the structure–property link. After training their GAN with style transfer loss and an additional loss to penalize mode collapse, they treat the noise vectors as input variables and optimize them with Bayesian optimization, using the generator to translate between design vectors and microstructure images. The authors choose to optimize microstructures for energy absorption, which can be evaluated from images using coupled-wave analysis [130], providing them a convenient structure–property link that would be more challenging if optimizing for other objectives.

More recently, Fokina et al. [104] adapt StyleGAN [7] to the microstructure image synthesis domain. Other studies impose the "style" of generated microstructure images through enforcement of physical properties. For example, Chen and Liu [89] propose an approach to generate random heterogeneous material microstructure images using a cGAN conditioned on target images. Their cGAN uses an augmented loss based on the matching of perimeter, volume, and Euler characteristics. DGM studies in microstructure design have also used other advanced methods introduced in computer vision such as super-resolution [58] and image translation [4,67,124].

Although 2D images are a widely used means to represent microstructures, Zhang et al. [65] expand the microstructure reconstruction task to the 3D domain. Their ScaffoldGAN method generates scaffold materials to mimic real-world examples of human bone scaffolds as well as foam metal scaffolds in both 3D and 2D data. They employ the conventional GAN approach with style loss for better visual similarity between generated scaffolds and real ones. However, the authors additionally introduce a novel "structural loss" term to specifically address spatial coherence, a limitation of GANs in emulating scaffolds. This additional loss helps them achieve better results that mimic the features of the data more realistically. Other studies also consider DGMs on 3D microstructures, such as Mosser et al. [66] and Liu et al. [68].

5.2.1 Photonics and Phononics. Within microstructure design, the design of materials with photonic or phononic properties is an area of interest in numerous industries including sensing, communications, and display technology. The work by Yang et al. [64] is an example of a subfield of generative microstructure design targeting the development of microstructures with particular photonic or phononic properties. Molesky et al. [131] also provide a review of inverse design in nanophotonics.

In the photonics and phononics subfield, performance evaluations such as the one used in Ref. [64] are frequently used to incorporate performance into DGMs. For example, the technique of optimization using learned mappings from the latent space of a trained autoencoder to the property space has been employed in several papers. Li et al. [81], Liu et al. [82], and Wang et al. [105] train an autoencoder, VAE, and Gaussian Mixture VAE [132], respectively, on images of microstructures. They map the latent variables to the property space using a DNN, CNN, and Gaussian Process Regressor, respectively. Li et al. [81] directly optimize the properties using the DNN, while Liu et al. [82] and Wang et al. [105] optimized using a genetic algorithm.

Although optimizing latent variables is a common approach, several other DGMs have been proposed for photonics/phononics design that optimize performance in other ways. For instance, Malkiel et al. [56] uniquely used a parametric representation and a bidirectional DNN to simultaneously learn a bidirectional mapping between nanostructures and the property space. Ma et al. [91] proposed a novel VAE-based framework consisting of feature extraction, prediction, recognition, and generation networks. The proposed method is capable of both forward prediction of properties based on metamaterial structure as well as the inverse (generative) prediction task based on properties. Furthermore, the framework supports self-supervised learning where the model trains on unlabeled metamaterial pattern images without corresponding property labels. The authors also demonstrated transfer learning to other microstructure shape classes as well as the inverse design of multiple microstructures forming a meta-mirror with desired properties. DGMs have also been applied to nano-scale photonic devices, such as an optic broadband power splitter as presented in Tang et al. [88].

5.2.2 Unit Cell-Based Metamaterials. Research in generative design for microstructures has also focused on the development of unit cell structures for metamaterials. For instance, Wang et al. [32] fitted a VAE to metamaterial unit cells and demonstrated that latent space parameters can reflect the physical properties of the unit cells. They then demonstrated several downstream tasks using their VAE, such as diverse subset selection and generation, targeted generation to match desired stiffness matrix values, and metamaterial family design. The authors also demonstrate several 2D macro-optimization runs to design arrays of unit cells to match macro-level deflection targets in an approach similar to classic TO. Xue et al. [83] trained a VAE to generate unit cells



Fig. 3 Deep generative models are often employed to generate 3D metamaterial unit cells

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

and then performed Bayesian optimization in the latent space of the VAE to attain desired macroscopic elastic properties.

5.3 2D Shape Synthesis. Designing features with critical geometric considerations is a task that appears in several engineering fields such as aerospace and automobile design (Fig. 4). DGMs have been widely applied to these problems. Within aerospace, the design of an airfoil, which is the cross-sectional shape of a wing, is of particular interest to many researchers. Airfoils have a wide variety of uses in engineering domains such as propeller, rotor, and turbine blade design. Since airfoil performance parameters are of key interest, most of the research focuses on performance-aware conditional generation. For example, Yilmaz and German [74] apply a cGAN to the airfoil design problem, conditioning their network on various stall parameters. While the overwhelming majority of research in this domain uses DGMs trained to learn shape parameterization based on spline interpolation of Cartesian points, equation-based parameterization is also sometimes used. For example, Li et al. [94] proposed a performance-aware RL framework where the RL agent learns the optimal equation coefficients using proximal policy optimization [133].

Other works generalize their applications to general 2D shape synthesis. For example, Chen and Fuge [75] proposed Bézier GAN, a framework that learns shape representations through Bézier curves, featuring InfoGAN style conditioning [17]. Chen et al. [76] expanded on this work with Bayesian Optimization to maximize lift/drag ratio. Classic test data for 2D shape synthesis methods are the UIUC airfoil dataset as well as artificially designed shapes like superformulas [134]. While the emphasis of certain papers is on shapes or airfoils themselves, several papers primarily propose methodology advancements that they choose to demonstrate on the 2D shape synthesis domain. For example, Chen and Fuge [33] addressed the challenging problem of multicomponent design generation using a GAN to synthesize parts using inter-part dependencies. The authors assume inter-part dependencies in a design are known and propose modeling them using directed acyclic graphs. They proposed a hierarchical adaptation of the InfoGAN using a single discriminator for the entire design and one generator/auxiliary network pair for each part in the design, which they term the hierarchical GAN. The method is tested on a medley of synthetic datasets generated using BézierGAN [75] and demonstrated to form meaningful and interpretable latent spaces. Although the paper assumes that part dependency graphs for the object class are well defined, this article takes a significant step toward multicomponent design synthesis and interpretable GANs.

Several other papers use the 2D shape synthesis domain to address the challenges of performance evaluation in DGMs. Dering et al. [95], for example, apply an iterative retraining approach to boat sketches using an adaptation of the long short-term memory-based Sketch-RNN [135] as the generator. To evaluate candidate designs, the performance is scored using a simulated environment in a game engine, within which the "behavior" (motion) of the design is learned.

A recent work by Chen and Ahmed [15] addressed both performance evaluation and design novelty through their proposed performance augmented diverse GAN (PaDGAN) framework. Conventional GANs are trained to mimic the design space they are trained in and, as a consequence, are penalized for generating novel designs. PaDGAN expands on the conventional GAN architecture by modeling design performance and design diversity using a determinantal point process kernel [136]. By promoting diversity, PaDGAN directly addresses mode collapse in GANs too (see Sec. 2.1). The authors demonstrated that PaDGAN is capable of generating high-quality and previously unseen novel designs for the UIUC airfoil database, using the lift-to-drag ratio as the quality metric. Chen and Ahmed [137] extended this work to multi-objective generation.



Fig. 4 Airfoil synthesis using DGMs usually entails employing 2D shape generation approaches

5.4 3D Shape Synthesis. 3D object generation using deep learning is an active field in computer science, with significant research efforts being dedicated to generating realistic-looking shapes and objects (Fig. 5). 3D shapes and objects are typically represented by voxels, point clouds, or meshes. Advancements in 3D shape synthesis in the computer graphics community have leaned heavily on GANs or autoencoders, as well as other machine learning advancements like RNNs, transformers, and GNNs. This research is tangentially relevant to engineering design but is typically focused more on visual appearance and aesthetics rather than design considerations like functional performance derived from simulations or manufacturability. Hence, we do not discuss these methods in detail and opt to list a few of the many noteworthy papers in this field from 2016 to 2021 for the reader to further explore at their discretion [138–143]. It is important to note that many of these proposed DGM architectures could be adapted to the design domain too. Despite the prominence of 3D object generation papers in the computer graphics community, a few have arisen within the engineering design community as well. In an early work, for example, Brock et al. [84] implement a 3D cVAE to reconstruct and interpolate between voxelized models from the ModelNet-10 Dataset.

The majority of 3D shape synthesis work in engineering design, however, has implemented some kind of design performance consideration. Zhang et al. [92] proposed the use of a genetic algorithm to optimize latent space design embeddings of a trained VAE variant called the variational shape learner [144], which they demonstrate on the optimization of 3D aircraft models. Shu et al. [69] take an iterative retraining approach by retraining a GAN on high-performance models evaluated using CFD evaluation. The authors applied the proposed method to point cloud aircraft models from ShapeNet and used minimization of aerodynamic drag as the performance objective of choice. The authors used a standard GAN loss and also a discriminator architecture from Ref. [145]. Heyrani Nobari et al. [77] introduced another self-augmentation approach to train on skewed or sparse datasets. They coupled it with a "range loss" to encourage designs to comply with design constraints related to parameter bounds and apply their approach to generate 3D aircraft models.

5.5 Other Applications. In this section, we discuss several application domains for DGMs with only a few relevant papers identified in our search.

5.5.1 Manufacturability. Synthesizing manufacturable designs is an important, albeit less-explored area in DGMs. Greminger [100] proposed approaching the problem of manufacturability using an MSG-GAN architecture [146] adapted to the 3D domain. The author synthetically generated topologies that are manufacturable by a three-axis mill, trained the GAN to generate similar topologies, and then optimized generated topologies using TO. A key challenge in GAN-based synthesis of manufacturable designs is a lack of annotated datasets with design and manufacturing process details.



Fig. 5 DGM models, such as Range-GAN [77], can be used to generate 3D models of aircrafts with constraints

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

5.5.2 Fluid Flow Shapes. DGMs have also been used to generate flow shapes. For example, Lee et al. [96] applied a double deep Q-network (DoubleDQN) [147] RL framework to learn the geometry of a microfluidic channel to generate target flow shapes. The authors also proposed methods in which a human designer can participate in the design process alongside the RL agent to impart design knowledge or constraints. Other approaches that integrate humans into the design generation process are discussed next.

5.5.3 Integrating Humans Into the Generation Process. One element of design that is often less emphasized in DGMs for design is human interpretation and design input during the generative process. Although datasets are often created using some human input, humans are not directly involved during the training process of most DGMs. Burnap et al. [90] noted that numerical performance measures often do not correspond to a human's perception of design quality and studied this phenomenon on automobile images generated through a conditional VAE. Supporting the involvement of humans during the training process is a potential approach to find the middle ground between the expertise and intuition of humans and the detailed information of datasets, which can enable different tasks. For example, Wang et al. [78] looked at the problem of generating samples that are visually pleasing for any given human subject. To do this, they train a modified version of the auxiliary classifier GAN [148], which classifies the type of image being generated at the same time as learning to generate images. The authors, however, do not opt for a cGAN architecture. Instead, they condition the GAN using electroencephalography (EEG) signals from human subjects. They train an encoder that takes EEG signals and transforms them into a set of input features for the GAN. By exposing human subjects to the images as they are being generated, the encoder introduces the human response to the design into the generation process. This effectively allows for a human subject to extract more visually pleasing samples from the GAN model. Several other studies such as Refs. [73,85,96] proposed frameworks that incorporate humans into the design process.

5.5.4 Kinematic Synthesis. In machine design, generating a mechanism to generate specific motion curves or transmit power is a common design task. Kinematic synthesis involves selecting the size, type, and configuration of mechanism components to achieve such a goal. Several works have applied DGMs to various kinematic synthesis problems. Deshpande and Purwar [85], for example, proposed a VAE-based method for planar linkage synthesis. They deterministically parameterized both coupler paths and linkages and then generated sample coupler paths from a variety of linkages (four-bar, slider-crank, Stephenson-type six-bar). In the first of their two studies, the authors trained a VAE to reconstruct trajectories. They then proposed a method in which humans can interact with the trained VAE to customize linkage paths to their design goals by visualizing the effect of various latent space perturbations and selecting one that matches their design vision. The authors then expand on their first study with a cVAE architecture, feeding in the linkage as the data samples and the coupler curve as the condition vector. In a similar work, Sharma and Purwar [86] explore spatial linkage synthesis of 5-SS mechanisms using VAEs. In other papers, the same authors take a look at paths themselves as guiding mediums for linkage mechanism design by first training models to encode paths, then using said models to search within a dataset of mechanisms to obtain solutions for the specific paths [93].

Other studies have approached the kinematic synthesis task through reinforcement learning. Vermeer et al. [107] proposed using deep Q-learning to synthesize planar linkage mechanisms to obtain mechanisms that are capable of producing straight lines, showing that RL can be effective in generating linkage mechanisms using machine learning.

5.5.5 Truss Design. Raina et al. [108] proposed an RL-based approach to truss design. In their approach, they learn from a dataset of sequential human design decisions [149]. They first trained an autoencoder to map trusses to and from a design embedding. Sequential design embeddings are then used to predict a heatmap of possible next design states using a supervised transition network. Interestingly, the authors choose to convert to and from images from their parametric representation when training the autoencoder and using decoded results. Finally, a rule-based agent was selected from possible next design steps. Even without knowledge of design metrics and performance, the RL agents were found to generate competitive designs compared to humans. Puentes et al. [109] expanded on this work by learning action-sequence heuristics instead of individual design actions. Raina et al. [97] further expanded on this work with a goal-based reinforcement learning agent.

5.5.6 Hierarchical Product Design Synthesis. Some works have applied DGMs to multi-component products, typically with a well-known hierarchical structure. Stump et al. [34], for example, proposed a unique approach that simultaneously combined RL, RNNs, grammars, and physics-based simulation. In their approach, the RNN learns to generate modular sailing crafts by sampling discrete selections from a predefined shape grammar. The training of the RNN is detailed in Ref. [110]. The control policy of the craft is then optimized using RL in a physics-based simulated sailing environment. Regenwetter et al. [87] explored the full synthesis of bicycles, a diverse class of products typically consisting of numerous hierarchical components. The authors demonstrated full generative synthesis of bikes using VAEs for both image and parametric representations. The authors note that their detailed design parameterization allows AI-generated designs to be physically fabricated.

5.5.7 Procedural Content Generation. Lopez et al.explored procedural content generation in a specific context for virtual reality (VR). In their work, they introduced a reinforcement learning model that can learn to generate 3D VR content, which users can explore in a VR environment [98]. In their approach, they specifically work toward generating manufacturing environments that are physically valid and feasible. In an extension of this work, Cunningham et al. extended the method to generate content for multiple contexts instead of just one at a time [99], which enables the integration of user-specific parameters.

6 Datasets

In this section, we present commonly used datasets that have been or have the potential to be used to train DGMs for data-driven design tasks. We provide a more detailed list online.[1] While the datasets listed are not comprehensive, we aim to note key types of datasets that have been commonly used in engineering design applications. We hope that researchers can create larger well-annotated datasets and make them public for the community to use.

6.1 Topology Optimization Datasets. Most papers on DGMs for topology optimization generate their own datasets, which are often not publicly available. The most common method for dataset generation is solid isotropic material with penalisation (SIMP), which has several publicly available software implementations. A few TO datasets, however, are open source. Sosnovik and Oseledets [52] generated a dataset of 10,000 artificially generated topologies providing both final and intermediate topologies from the optimization process.[2] Each included topology contains one 40x40 image of the topology generated at every stage of 100 steps of topology optimization. Hence, a total of one million images are provided. Nie et al. [71] provided the dataset used

[1] http://decode.mit.edu/datasets/
[2] https://github.com/ISosnovik/top

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

to train their TopologyGAN framework.[3] Unlike the aforementioned, this dataset consists only of final topologies but contains 49,078 generated topologies at a resolution of 64×128, generated from 42 unique boundary conditions. Both datasets discussed use the ToPy [150] implementation of SIMP.

6.2 Microstructure Datasets. Well-established technologies such as optical microscopy and scanning electron microscopy are often used to visualize material microstructures. As such, numerous datasets of microstructure scan images are publicly available, such as Refs. [151–153]. Numerous datasets are also available for the design of composite materials of various types, such as the NanoMine nanopolymer composite database,[4] which contains over 20,000 datapoints [154]. Compiled lists of materials science datasets[5] and synthetic microstructure datasets are also available. For example, Yang et al. [64] provided a trained GAN model that generates synthetic microstructure images.[6]

6.3 Design Geometry Datasets. The UIUC airfoil database[7] has been used as a case study in several generative design research frameworks [15,75,76]. The database details nearly 1600 real-world airfoil designs using coordinates of points on the surface. Since the original data provide inconsistent numbers of coordinates along the top and bottom surfaces, Chen et al. [76] proposed a method to standardize the data with B-spline interpolation over the airfoil, which is also used in other works [15,33].

6.4 3D Object Datasets. ShapeNet[8] [155] is one of the most commonly used 3D model datasets, consisting of over 51,300 3D models of 55 object categories. PartNet [156] expands on ShapeNet with fine-grained hierarchical semantic annotations for component parts of ShapeNet objects. Princeton ModelNet[9] [157] is another commonly used 3D model dataset consisting of 127,915 voxel-based 3D models of 662 object categories. Numerous works discussed in this review use ShapeNet or ModelNet models [69,77,84,92].

Kim et al. [158] introduced a dataset of 58,696 models of mechanical components from 68 classes called the mechanical components benchmark (MCB).[10] The MCB contains a hierarchical label tree grouping components into subclasses of different levels, such as Components → Fasteners → Nuts → Wingnuts, for example. Models are represented as point clouds, voxels, and 2D views.

6.5 CAD and CAD-Based Datasets. Willis et al. [159] introduced two datasets of Autodesk Fusion models.[11] One dataset, intended for reconstruction tasks, contains 8,625 models and the other, intended for segmentation, contains 35,680 models. The reconstruction dataset is particularly interesting for generative tasks, as it contains information governing the sequential CAD operation steps taken to generate a part.

Regenwetter et al. [87] introduced a dataset called BIKED[12] consisting of mixed data extracted from 4512 bicycle CAD models. The dataset includes bicycle assembly images, segmented subcomponent images, as well as "parametric" data. The "parametric" data consist of 2395 mixed-type design parameters describing both high-level and low-level characteristics. BIKED provides an advantage over conventional 3D object datasets for generative tasks in that synthesized designs contain the necessary parametric information to physically fabricate designs. BIKED's parametric data are used in Ref. [87] for bicycle synthesis and its image data in Ref. [160] for generating novel designs. The FRAMED dataset [161] expands on BIKED with structural performance data, such as weight, safety factors, and deflections under various loads for all 4512 models as well as artificially generated bicycle frames.[13]

6.6 Metamaterials Datasets. Wang et al. [32] introduced a dataset of 248,396 2D unit cells represented by 50x50 pixelated matrices. The unit cells have associated stiffness tensor components provided and can also be used for TO research. Chan et al. [162] introduced a dataset of 3000 3D isosurface unit cells sampled from 30 level-set functions, along with corresponding 3D elastic tensor components. Wang et al. [163] introduced a dataset of 795 unit cells generated from ten lattice models. Associated stiffness tensors are provided. The three datasets can be found here.[14]

6.7 Sketch Datasets. QuickDraw [164] is a sketch dataset of 50 million doodles from 345 categories.[15] The doodles were collected by Google from user-drawn sketches in an interactive sketching game. QuickDraw data are used in several works discussed [95,165]. Toh and Miller [166] introduced a dataset of 934 innovative milk frother design sketches with associated text descriptions that can potentially be used to train DGMs, perhaps factoring in natural language processing.[16]

6.8 Sequential Human Design Datasets. McComb et al. [149] provided a tabular truss design dataset taken from a truss design activity executed by 16 human teams.[17] Sequential design operations such as joint and member placement are recorded using geometric parameters such as joint coordinates, member size, and so on. Performance metrics such as safety factors and weights are also included. While this dataset can be used as a truss design dataset, it is primarily intended as a resource to study or mimic the human design process. We note that the Autodesk Fusion reconstruction dataset mentioned previously [159] can also be used for learning sequential design tasks.

7 Discussion, Challenges, and Future Work

When applying DGMs to engineering design, the "standard" objective of mimicking the training data is often insufficient, or even counterproductive. Instead, real designs are governed by specific objectives and constraints, often including novelty or creativity. Thus, different objectives, such as real-world performance metrics, novelty, and adherence to constraints, may make for better training objectives. We discuss the challenges with performance evaluation, constraint violation, and incorporation of novelty in the following sections, as well as pathways to potential solutions. We further discuss some general challenges for DGMs in engineering design, such as limited availability of data, lack of benchmark problems and metrics, and delayed adoption of cutting edge methods from different research communities. Our discussion is supported by observations from our literature review.

7.1 Design Performance Evaluation. Incorporating performance evaluation into DGMs is one of the key stepping stones toward practical applications of ML in design. Three major challenges in performance evaluation are fidelity, cost, and differentiability.

[3] https://github.com/zhenguonie/2020˙TopologyGAN
[4] https://github.com/tetherless-world/nanomine-ontology
[5] https://github.com/sedaoturak/data-resources-for-materials-science
[6] https://github.com/zyz293/GAN˙Materials˙Design
[7] https://m-selig.ae.illinois.edu/ads/coord˙database.html
[8] https://shapenet.org/
[9] https://modelnet.cs.princeton.edu/
[10] https://bit.ly/3ne4gwv
[11] https://github.com/AutodeskAILab/Fusion360GalleryDataset
[12] https://decode.mit.edu/projects/biked/
[13] https://decode.mit.edu/projects/framed/
[14] https://ideal.mech.northwestern.edu/research/software/
[15] https://github.com/googlecreativelab/quickdraw-dataset
[16] https://sites.psu.edu/creativitymetrics/2018/07/18/milkfrother/
[17] https://www.sciencedirect.com/science/article/pii/S2352340918302014?via%3Dihub

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

(1) Evaluation methods lacking fidelity may result in DGM-generated designs that do not meet specifications.
(2) Computational cost precludes compatibility with methods that heavily sample performance values.
(3) Lack of differentiability makes implementation into the training process difficult in any machine learning models that rely on gradient-based optimization.

Physical evaluation, which means building a product and testing its performance in the real-world, typically has the highest fidelity but is rarely adopted in DGMs due to its prohibitive cost. Qualitative human evaluation of designs has also been investigated in several papers [73,85,96], although the evaluation is typically not focused on performance. Medium-fidelity evaluations, such as numerical simulations, often deliver satisfactory fidelity, but can be costly and are rarely differentiable. Methods that do incorporate medium-fidelity performance evaluation like [62,69] do not rely on performance score gradients, and instead alternate (re)training and performance evaluation, typically doing this retraining a handful of times.

Low-cost evaluation methods like surrogate models can be worked into the training objective and are by far the most common performance evaluation method seen in DGMs [15,18,20,60,70,77,79]. Unfortunately, surrogate models can be brittle and generalize poorly to designs that differ from the data the surrogate was trained on. This is a particular concern when training data for surrogates are unevenly distributed across the performance space [77] and when novelty and performance are incorporated into training objectives for DGMs. Many different approaches have attempted to improve the performance of low-fidelity surrogate models. For example, self-supervised data augmentation [77] uses the DGM itself to generate samples in sparse regions (through optimization [137] or conditioning [77]), which then can be used to train the low-fidelity models to perform more accurately and in turn improve methods that rely on them for design generation. Multifidelity modeling is another promising approach, which involves generating surrogate models that augment a few costly high-fidelity samples with low-fidelity samples to attain higher fidelity surrogates with minimal expense.

Advancements have also been made in using machine learning to improve and accelerate medium-fidelity physics simulations such as finite element analysis [167–170] and computational fluid dynamics [170–173]. These works, although not yet robust enough to be easily applied as an alternative to high-fidelity simulations, provide a proof of concept for machine learning-based acceleration and even replacement of higher-fidelity simulations.

Numerous studies have also proposed crowdsourcing methods to evaluate synthesized data from generative machine learning methods [90,165,174] instead of physical or physics-based evaluation. Although the fidelity of crowdsourced evaluation is highly task and crowd expertise dependent, the high evaluation cost and lack of differentiability are fairly universal.

In summary, current DGMs are largely constrained in performance evaluation by an inherent fidelity versus cost tradeoff; however, several directions show promise in enabling faster and more accurate evaluation methods that may escape this limitation.

7.2 Feasibility, Constraints, and Manufacturability. One key component of design performance is obeying explicit design constraints, as well as implicit constraints such as physical feasibility and manufacturability. DGMs are difficult to rely upon for explicit design constraints since they are generally probabilistic and may generate completely invalid designs. This issue is a concern that many researchers have pointed to Refs. [33,103,175]. One potential solution is to develop inexpensive and reliable validation methods; however, this is a challenging task that may require significant human input.

Another underlying issue lies in design representation, which is discussed in detail in Sec. 3. Representations such as images often have no clear translation to representations with practical uses. Other representations, such as 3D models of various types can be realistically fabricated only using additive manufacturing, which precludes many design domains. For ML-generated designs to be physically fabricated and used in practical applications, the feasibility across this "domain gap" must be overcome [87,103]. Although a few works have investigated manufacturability [100], we observed that modeling it is not considered in most of the papers we reviewed. Other works have attempted to apply DGMs to parameterizations that encode similar parametric design data to what would be used in manufacturing drawings. However, the same works identify that parametric data of this sort is challenging to learn and generate [87].

7.3 Creativity and Novelty. Although creativity and novelty are essential aspects of the classic design process, DGMs rarely explicitly consider either. Most DGMs learn to mimic the data covering the existing and already explored portions of the design space. While this emulative behavior is helpful for maintaining realism and ensuring sample quality, it incentivizes DGMs against generating creative or novel designs [176]. For DGMs to progress toward more human-like design, advancements must be made in modeling creativity as well as in developing architectures that promote creativity.

Several recent works [15,137,160] have proposed methods to encourage creativity and novelty in DGMs. In their framework, CreativeGAN, Nobari et al. focused on identifying novelty and guiding DGMs toward such behavior by directly introducing novel features into typical designs, thereby expanding the design space and novelty of the DGM's data. Creativity in machine learning has also been explored outside of the design community. For example, creative adversarial networks (CAN) [176] introduce entropy into the training to encourage the generation of surprising images. Readers are directed to Franceschelli et al.'s survey on the topic for more detail [177].

7.4 Evaluating Model Performance and Benchmark Problems. Readers may have noted that many of the works discussed have nearly identical methodologies, although they are applied to a specific dataset and only compare their work in this respect with a few other baselines. This may be attributed to a large variety of design applications compared to other domains such as computer vision and a tendency of design researchers to find solutions to their particular problem, instead of finding generalizable solutions to many. However, this makes any measure of "state-of-the-art" algorithm dependent on specific applications. Without a good understanding of state-of-the-art methods that are broadly applicable across engineering domains, practitioners will struggle to select models for practical deployment. While some works in the design community have specifically introduced benchmark problems and datasets [87,161], there is still a need for larger, higher-quality, and more numerous datasets and benchmarks.

The difficulty of establishing a state-of-the-art lies in the lack of benchmark problems and performance metrics within the DGM field and within design automation as a whole. Design datasets tend to be small, restricted in domain, and sparse in distribution as we discuss in Sec. 7.5. Furthermore, many methods are developed on proprietary datasets. Finally, the hugely different design representation methods across the field make establishing standardized model performance benchmarks difficult, even if there were good datasets upon which to do so. These problems are noted by many authors in the field as well, who find it difficult to compare their approach with existing ones [60,175].

7.5 Data Limitations and Quality. Data sparsity is one of the greatest challenges facing the data-driven design community and is a particular concern for researchers developing data-hungry DGMs. Broadly, there are three problems when it comes to the data. The first of these is a general lack of data in many design domains.

ded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

Although datasets continue to cover more and more design fields, there still are many that lack publicly available data. There are also data representations that are underrepresented in the current datasets, notably graphs as discussed in Sec. 3. The second key data-related problem that designers face in applying DGMs is the insufficient size of current datasets. Many of the latest breakthroughs in deep learning, notably in computer vision and natural language processing, have owed their success to very large models that are notoriously data-hungry, requiring millions or billions of training examples, and cannot be properly utilized with datasets of smaller size. Focusing resources on the generation of very large publicly available datasets or effective data augmentation methods would open doors for the design community to leverage larger deep models. The last major limitation in existing data is sparsity and bias. These problems are common in many design datasets. How the data are distributed in the performance space can cause some challenges for many DGMs, especially in inverse design problems [77]. Diversity is important to ensure the data cover the design space as evenly as possible to avoid bias in the data. Keeping diversity and bias avoidance in mind will help researchers generate better datasets.

We highly encourage researchers to develop diverse datasets, publicly release these datasets, and establish benchmark problems for future works. We subsequently encourage researchers to test their methods on other researchers' data, publicly release their benchmarking results, and acknowledge state-of-the-art methods when possible.

7.6 Other Deep Generative Models. So far our discussion has been focused on the VAE-, GAN-, and RL-based approaches, which have dominated the field of DGMs in engineering design. Transformer-based sequential models were initially developed for natural language processing and generation or translation of text; however, researchers have shown that the scalability of these models allows for them to be used for the generation of complex data in a sequential manner [178–181]. Furthermore, these models have an unprecedented ability to be mixed with language processing. This may enable human language-based conditioning and control over the generation process of data [179,180], enabling deployment for a much wider user base and a much wider set of tasks. The applications of transformer-based models as well as their scalability may appeal to researchers in the design community. Despite this, it is important to note that these models are particularly large and therefore require very large datasets, further highlighting the need for better datasets.

8 Conclusion

In this review, we discussed the applications of DGMs across engineering design fields. To give readers a sense of the tools and methods available, we began our review with an overview of DGMs typically used for engineering design problems, emphasizing GANs, variational autoencoders (VAEs), and RL. To help weigh different design representation methods, we then discussed the strengths and weaknesses of common parameterization methods. To inform readers about existing work outside of their subdomain, we then collected and reviewed 63 papers from a variety of different engineering design subdisciplines that directly propose DGMs. To make readers aware of the datasets available on which to develop and test data-driven design methods, we review commonly used datasets in the field. Finally, to provide inspiration, we discuss key challenges and limitations currently seen across the field and highlight possible solution pathways.

Conflict of Interest

There are no conflicts of interest.

References

[1] Chakrabarti, A., Shea, K., Stone, R., Cagan, J., Campbell, M., Hernandez, N. V., and Wood, K. L., 2011, "Computer-Based Design Synthesis Research: An Overview," ASME J. Comput. Inf. Sci. Eng., **11**(2), p. 021003.

[2] Deng, L., and Yu, D., 2014, "Deep Learning: Methods and Applications," Found. Trends Signal Process., **7**(3–4), pp. 197–387.

[3] Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y., 2014, "Generative Adversarial Nets," Proceedings of the 27th International Conference on Neural Information Processing Systems – Volume 2, NIPS'14, Montréal, Quebec, Canada, MIT Press, pp. 2672–2680.

[4] Zhu, J.-Y., Park, T., Isola, P., and Efros, A. A., 2017, "Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks," 2017 IEEE International Conference on Computer Vision (ICCV), Venice, Italy, pp. 2242–2251.

[5] Choi, Y., Uh, Y., Yoo, J., and Ha, J.-W., 2020, "Stargan V2: Diverse Image Synthesis for Multiple Domains," 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Virtual, pp. 8185–8194.

[6] Karras, T., Laine, S., Aittala, M., Hellsten, J., Lehtinen, J., and Aila, T., 2020, "Analyzing and Improving the Image Quality of Stylegan," 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Virtual, pp. 8107–8116.

[7] Karras, T., Laine, S., and Aila, T., 2019, "A Style-Based Generator Architecture for Generative Adversarial Networks," 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, pp. 4396–4405.

[8] Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B., and Bharath, A. A., 2018, "Generative Adversarial Networks: An Overview," IEEE Signal Process. Mag., **35**(1), pp. 53–65.

[9] Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., and Chen, X., 2016, "Improved Techniques for Training GANS," Proceedings of the 30th International Conference on Neural Information Processing Systems, NIPS'16, Barcelona, Spain, pp. 2234–2242.

[10] Arjovsky, M., and Bottou, L., 2017, "Towards Principled Methods for Training Generative Adversarial Networks," arXiv:1701.04862.

[11] Arjovsky, M., Chintala, S., and Bottou, L., 2017, "Wasserstein Generative Adversarial Networks," Proceedings of the 34th International Conference on Machine Learning—Volume 70, ICML'17, Long Beach, CA, pp. 214–223.

[12] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., and Courville, A., 2017, "Improved Training of Wasserstein GANS," Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17, Long Beach, CA, pp. 5769–5779.

[13] Srivastava, A., Valkov, L., Russell, C., Gutmann, M. U., and Sutton, C., 2017, "Veegan: Reducing Mode Collapse in Gans Using Implicit Variational Learning," Advances in Neural Information Processing Systems, Long Beach, CA, pp. 3308–3318.

[14] Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., Chen, X., and Chen, X., 2016, "Improved Techniques for Training Gans," Advances in Neural Information Processing Systems, Barcelona, Spain, pp. 2234–2242.

[15] Chen, W., and Ahmed, F., 2021, "Padgan: Learning to Generate High-Quality Novel Designs," ASME J. Mech. Des., **143**(3), p. 031703.

[16] Mirza, M., and Osindero, S., 2014, "Conditional Generative Adversarial Nets," arXiv:1411.1784.

[17] Chen, X., Duan, Y., Houthooft, R., Schulman, J., Sutskever, I., and Abbeel, P., 2016, "Infogan: Interpretable Representation Learning by Information Maximizing Generative Adversarial Nets," Proceedings of the 30th International Conference on Neural Information Processing Systems, Barcelona, Spain, pp. 2180–2188.

[18] Dong, Y., Li, D., Zhang, C., Wu, C., Wang, H., Xin, M., Cheng, J., and Lin, J., 2020, "Inverse Design of Two-Dimensional Graphene/h-BN Hybrids by a Regressional and Conditional GAN," Carbon, **169**, pp. 9–16.

[19] Ding, X., Wang, Y., Xu, Z., Welch, W. J., and Wang, Z. J., 2021, "CcGAN: Continuous Conditional Generative Adversarial Networks for Image Generation," 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, New Orleans, LA, May 3–7.

[20] Heyrani Nobari, A., Chen, W., and Ahmed, F., 2021, "PcDGAN: A Continuous Conditional Diverse Generative Adversarial Network for Inverse Design," Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual, August.

[21] Kingma, D. P., and Welling, M., 2013, "Auto-Encoding Variational Bayes," arXiv:1312.6114.

[22] Kullback, S., and Leibler, R. A., 1951, "On Information and Sufficiency," Ann. Math. Statist., **22**(1), pp. 79–86.

[23] Kingma, D. P., and Welling, M., 2019, "An Introduction to Variational Autoencoders," Found. Trends Mach. Learning, **12**(4), pp. 307–392.

[24] Sohn, K., Yan, X., and Lee, H., 2015, "Learning Structured Output Representation Using Deep Conditional Generative Models," Proceedings of the 28th International Conference on Neural Information Processing Systems – Volume 2, NIPS'15, Montréal, Quebec, Canada, MIT Press, pp. 3483–3491.

[25] Kaelbling, L. P., Littman, M. L., and Moore, A. W., 1996, "Reinforcement Learning: A Survey," J. Artif. Int. Res., **4**(1), pp. 237–285.

[26] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., and Ostrovski, G., 2015, "Human-Level Control Through Deep Reinforcement Learning," Nature, **518**(7540), pp. 529–533.

aded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UIBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

[27] Daneshmand, M., Helmi, A., Avots, E., Noroozi, F., Alisinanoglu, F., Arslan, H. S., Gorbova, J., Haamer, R. E., Ozcinar, C., and Anbarjafari, G., 2018,"3D Scanning: A Comprehensive Survey," arXiv:1801.08863.
[28] Remondino, F., 2003, "From Point Cloud to Surface: The Modeling and Visualization Problem," International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, Tarasp-Vulpera, Switzerland.
[29] Ranjan, A., Bolkart, T., Sanyal, S., and Black, M. J., 2018, "Generating 3D Faces Using Convolutional Mesh Autoencoders," Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, pp. 704–720.
[30] Cheng, S., Bronstein, M., Zhou, Y., Kotsia, I., Pantic, M., and Zafeiriou, S., 2019, "Meshgan: Non-Linear 3D Morphable Models of Faces," arXiv:1903.10384.
[31] Zhang, Z., Wang, Y., Jimack, P. K., and Wang, H., 2020, "Meshingnet: A New Mesh Generation Method Based on Deep Learning," International Conference on Computational Science, Amsterdam, Netherlands, Springer, pp. 186–198.
[32] Wang, L., Chan, Y.-C., Ahmed, F., Liu, Z., Zhu, P., and Chen, W., 2020, "Deep Generative Modeling for Mechanistic-Based Learning and Design of Metamaterial Systems," Comput. Methods Appl. Mech. Eng., **372**, p. 113377.
[33] Chen, W., and Fuge, M., 2019, "Synthesizing Designs With Interpart Dependencies Using Hierarchical Generative Adversarial Networks," ASME J. Mech. Des., **141**(11), p. 111403.
[34] Stump, G. M., Miller, S. W., Yukish, M. A., Simpson, T. W., and Tucker, C., 2019, "Spatial Grammar-Based Recurrent Neural Network for Design Form and Behavior Optimization," ASME J. Mech. Des., **141**(12), p. 124501.
[35] Cao, Weijuan, Robinson, Trevor, Hua, Yang, Boussuge, Flavien, Colligan, Andrew, and Pan, Wanbin, 2020, "Graph Representation of 3D CAD Models for Machining Feature Recognition With Deep Learning," 46th Design Automation Conference (DAC) of International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Virtual, p. V11AT11A003.
[36] Yang, W., Ding, H., Zi, B., and Zhang, D., 2017, "New Graph Representation for Planetary Gear Trains," ASME J. Mech. Des., **140**(1), p. 012303.
[37] Hsu, C.-H., and Lam, K.-T., 1992, "A New Graph Representation for the Automatic Kinematic Analysis of Planetary Spur-Gear Trains," ASME J. Mech. Des., **114**(1), pp. 196–200.
[38] Lee, J. Y., and Kim, K., 1996, "Geometric Reasoning for Knowledge-Based Parametric Design Using Graph Representation," Comput.-Aided Des., **28**(10), pp. 831–841.
[39] Coatanéa, Eric, Nonsiri, Sarayut, Christophe, Francois, and Mokammel, Faisal, 2014, "Graph Based Representation and Analyses for Conceptual Stages," 34th Computers and Information in Engineering Conference of International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Buffalo, NY, p. V01AT02A071.
[40] Patalano, S., Vitolo, F., and Lanzotti, A., 2013, "A Graph-Based Software Tool for the CAD Modeling of Mechanical Assemblies," International Joint Conference on Computer Vision, Beijing, China.
[41] Henaff, M., Bruna, J., and LeCun, Y., 2015, "Deep Convolutional Networks on Graph-Structured Data," arxiv preprint.
[42] Yun, S., Jeong, M., Kim, R., Kang, J., and Kim, H. J., 2019, "Graph Transformer Networks," Advances in Neural Information Processing Systems, Vancouver, British Columbia, Canada.
[43] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y., 2018, "Graph Attention Networks," International Conference on Learning Representations, Vancouver, British Columbia, Canada.
[44] Vashishth, S., Sanyal, S., Nitin, V., and Talukdar, P., 2019, "Composition-Based Multi-Relational Graph Convolutional Networks," International Conference on Learning Representations, New Orleans, LA.
[45] Li, Y., Tarlow, D., Brockschmidt, M., and Zemel, R., 2017, "Gated Graph Sequence Neural Networks," arxiv preprint.
[46] Liao, R., Li, Y., Song, Y., Wang, S., Hamilton, W., Duvenaud, D. K., Urtasun, R., and Zemel, R., 2019, "Efficient Graph Generation with Graph Recurrent Attention Networks," Advances in Neural Information Processing Systems, Vancouver, British Columbia, Canada, Curran Associates, Inc.
[47] Bojchevski, A., Shchur, O., Zügner, D., and Günnemann, S., 2018, "Netgan: Generating Graphs Via Random Walks," International Conference on Machine Learning, Stockholm, Sweden, pp. 609–618.
[48] You, J., Ying, R., Ren, X., Hamilton, W., and Leskovec, J., 2018, "GraphRNN: Generating Realistic Graphs With Deep Auto-Regressive Models," International Conference on Machine Learning, Stockholm, Sweden, pp. 5708–5717.
[49] Li, Y., Vinyals, O., Dyer, C., Pascanu, R., and Battaglia, P., 2018, "Learning Deep Generative Models of Graphs," arxiv preprint.
[50] Cao, N. D., and Kipf, T., 2018, "Molgan: An Implicit Generative Model for Small Molecular Graphs," arxiv preprint.
[51] You, J., Liu, B., Ying, R., Pande, V., and Leskovec, J., 2018, "Graph Convolutional Policy Network for Goal-Directed Molecular Graph Generation," Proceedings of the 32nd International Conference on Neural Information Processing Systems, Montréal, Quebec, Canada, pp. 6412–6422.
[52] Sosnovik, I., and Oseledets, I., 2019, "Neural Networks for Topology Optimization," Russ. J. Numer. Anal. Math. Model., **34**(4), pp. 215–223.
[53] Behzadi, M. M., and Ilieş, H. T., 2021, "Real-Time Topology Optimization in 3D Via Deep Transfer Learning," Comput. Aided Des., **135**, p. 103014.
[54] Keshavarzzadeh, V., Alirezaei, M., Tasdizen, T., and Kirby, R. M., 2021, "Image-Based Multiresolution Topology Optimization Using Deep Disjunctive Normal Shape Model," Comput. Aided Des., **130**, p. 102947.
[55] Cang, R., Yao, H., and Ren, Y., 2019, "One-Shot Generation of Near-Optimal Topology Through Theory-Driven Machine Learning," Comput. Aided Des., **109**, pp. 12–21.
[56] Malkiel, I., Mrejen, M., Nagler, A., Arieli, U., Wolf, L., and Suchowski, H., 2018, "Plasmonic Nanostructure Design and Characterization Via Deep Learning," Light: Sci. Appl., **7**(1), pp. 1–8.
[57] Li, X., Zhang, Y., Zhao, H., Burkhart, C., Brinson, L. C., and Chen, W., 2018, "A Transfer Learning Approach for Microstructure Reconstruction and Structure-Property Predictions," Sci. Rep., **8**(1), pp. 1–13 .
[58] Jung, J., Na, J., Park, H. K., Park, J. M., Kim, G., Lee, S., and Kim, H. S., 2021, "Super-Resolving Material Microstructure Image Via Deep Learning for Microstructure Characterization and Mechanical Behavior Analysis," npj Comput. Mater., **7**(1), pp. 1–11.
[59] Li, B., Huang, C., Li, X., Zheng, S., and Hong, J., 2019, "Non-Iterative Structural Topology Optimization Using Deep Learning," Comput.-Aided Des., **115**, pp. 172–180.
[60] Rawat, S., and Shen, M. H., 2019, "Application of Adversarial Networks for 3D Structural Topology Optimization," SAE Technical Paper, Technical Report.
[61] Oh, S., Jung, Y., Lee, I., and Kang, N., 2018, "Design Automation by Integrating Generative Adversarial Networks and Topology Optimization," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Quebec City, Quebec, Canada, p. V02AT03A008.
[62] Oh, S., Jung, Y., Kim, S., Lee, I., and Kang, N., 2019, "Deep Generative Design: Integration of Topology Optimization and Generative Models," ASME J. Mech. Des., **141**(11), p. 111405.
[63] Tan, R. K., Zhang, N. L., and Ye, W., 2020, "A Deep Learning–Based Method for the Design of Microstructural Materials," Struct. Multidiscipl. Optim., **61**(4), pp. 1417–1438.
[64] Yang, Z., Li, X., Catherine Brinson, L., Choudhary, A. N., Chen, W., and Agrawal, A., 2018, "Microstructural Materials Design Via Deep Adversarial Learning Methodology," ASME J. Mech. Des., **140**(11), p. 111416.
[65] Zhang, H., Yang, L., Li, C., Wu, B., and Wang, W., 2021, "Scaffoldgan: Synthesis of Scaffold Materials Based on Generative Adversarial Networks," Comput. Aided Des., **138**, p. 103041.
[66] Mosser, L., Dubrule, O., and Blunt, M. J., 2017, "Reconstruction of Three-Dimensional Porous Media Using Generative Adversarial Neural Networks," Phys. Rev. E, **96**(4), p. 043309.
[67] Lee, J.-W., Goo, N. H., Park, W. B., Pyo, M., and Sohn, K.-S., 2021, "Virtual Microstructure Design for Steels Using Generative Adversarial Networks," Eng. Rep., **3**(1), p. e12274 .
[68] Liu, S., Zhong, Z., Takbiri-Borujeni, A., Kazemi, M., Fu, Q., and Yang, Y., 2019, "A Case Study on Homogeneous and Heterogeneous Reservoir Porous Media Reconstruction by Using Generative Adversarial Networks," Energy Procedia, **158**, pp. 6164–6169.
[69] Shu, D., Cunningham, J., Stump, G., Miller, S. W., Yukish, M. A., Simpson, T. W., and Tucker, C. S., 2020, "3D Design Using Generative Adversarial Networks and Physics-Based Validation," ASME J. Mech. Des., **142**(7), p. 071701.
[70] Sharpe, C., and Seepersad, C. C., 2019, "Topology Design With Conditional Generative Adversarial Networks," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Anaheim, CA, p. V02AT03A062.
[71] Nie, Z., Lin, T., Jiang, H., and Kara, L. B., 2021, "Topologygan: Topology Optimization Using Generative Adversarial Networks Based on Physical Fields Over the Initial Domain," ASME J. Mech. Des., **143**(3), p. 031715.
[72] Yu, Y., Hur, T., Jung, J., and Jang, I. G., 2019, "Deep Learning for Determining a Near-Optimal Topological Design Without Any Iteration," Struct. Multidiscipl. Optim., **59**(3), pp. 787–799.
[73] Valdez, S., Seepersad, C., and Kambampati, S., 2021, "A Framework for Interactive Structural Design Exploration," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, IDETC-21, Virtual.
[74] Yilmaz, E., and German, B., 2020, "Conditional Generative Adversarial Network Framework for Airfoil Inverse Design," AIAA AVIATION 2020 FORUM, Virtual, p. 3185.
[75] Chen, W., and Fuge, M., 2018, "Béziergan: Automatic Generation of Smooth Curves From Interpretable Low-Dimensional Parameters," arXiv:1808.08871.
[76] Chen, W., Chiu, K., and Fuge, M., 2019, "Aerodynamic Design Optimization and Shape Exploration Using Generative Adversarial Networks," AIAA Scitech 2019 Forum, San Diego, CA, p. 2351.
[77] Heyrani Nobari, A., Chen, W. W., and Ahmed, F., 2022, "Range-GAN: Design Synthesis Under Constraints Using Conditional Generative Adversarial Networks," ASME J. Mech. Des., **144**(2), p. 021708.
[78] Wang, P., Peng, D., Li, L., Chen, L., Wu, C., Wang, X., Childs, P., and Guo, Y., 2019, "Human-in-the-Loop Design With Machine Learning," Proc. Design Soc. Int. Conf. Eng. Design, **1**(1), pp. 2577–2586.
[79] Guo, T., Lohan, D. J., Cang, R., Ren, M. Y., and Allison, J. T., 2018, "An Indirect Design Representation for Topology Optimization Using Variational Autoencoder and Style Transfer," 2018 AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference, Kissimmee, FL, p. 0804.
[80] Cang, R., Li, H., Yao, H., Jiao, Y., and Ren, Y., 2018, "Improving Direct Physical Properties Prediction of Heterogeneous Materials From Imaging Data Via Convolutional Neural Network and a Morphology-Aware Generative Model," Comput. Mater. Sci., **150**, pp. 212–221.
[81] Li, X., Ning, S., Liu, Z., Yan, Z., Luo, C., and Zhuang, Z., 2020, "Designing Phononic Crystal With Anticipated Band Gap Through a Deep Learning

ded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2hAAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UfBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

Based Data-Driven Method," Comput. Methods Appl. Mech. Eng., **361**, p. 112737.
[82] Liu, Z., Raju, L., Zhu, D., and Cai, W., 2020, "A Hybrid Strategy for the Discovery and Design of Photonic Structures," IEEE J. Emerg. Sel. Top. Circuits Syst., **10**(1), pp. 126–135 .
[83] Xue, T., Wallin, T. J., Menguc, Y., Adriaenssens, S., and Chiaramonte, M., 2020, "Machine Learning Generative Models for Automatic Design of Multi-Material 3D Printed Composite Solids," Extreme Mech. Lett., **41**, p. 100992.
[84] Brock, A., Lim, T., Ritchie, J. M., and Weston, N., 2016, "Context-Aware Content Generation for Virtual Environments," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Charlotte, NC, p. V01BT02A045.
[85] Deshpande, S., and Purwar, A., 2019, "Computational Creativity Via Assisted Variational Synthesis of Mechanisms Using Deep Generative Models," ASME J. Mech. Des., **141**(12), p. 121402.
[86] Sharma, S., and Purwar, A., 2020, "Path Synthesis of Defect-Free Spatial 5-SS Mechanisms Using Machine Learning," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Virtual, p. V010T10A034.
[87] Regenwetter, L., Curry, B., and Ahmed, F., 2022, "BIKED: A Dataset for Computational Bicycle Design With Machine Learning Benchmarks," ASME J. Mech. Des., **144**(3), p. 031706.
[88] Tang, Y., Kojima, K., Koike-Akino, T., Wang, Y., Wu, P., Tahersima, M., Jha, D., Parsons, K., and Qi, M., 2020, "Generative Deep Learning Model for a Multi-Level Nano-Optic Broadband Power Splitter," 2020 Optical Fiber Communications Conference and Exhibition (OFC), San Diego, CA, IEEE, pp. 1–3.
[89] Chen, H., and Liu, X., 2021, "Geometry Enhanced Generative Adversarial Networks for Random Heterogeneous Material Representation," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, IDETC-21, Virtual.
[90] Burnap, A., Liu, Y., Pan, Y., Lee, H., Gonzalez, R., and Papalambros, P. Y., 2016, "Estimating and Exploring the Product Form Design Space Using Deep Generative Models," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Charlotte, NC, p. V02AT03A013.
[91] Ma, W., Cheng, F., Xu, Y., Wen, Q., and Liu, Y., 2019, "Probabilistic Representation and Inverse Design of Metamaterials Based on a Deep Generative Model With Semi-Supervised Learning Strategy," Adv. Mater., **31**(35), p. 1901111.
[92] Zhang, W., Yang, Z., Jiang, H., Nigam, S., Yamakawa, S., Furuhata, T., Shimada, K., and Kara, L. B., 2019, "3D Shape Synthesis for Conceptual Design and Optimization Using Variational Autoencoders," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Anaheim, CA, p. V02AT03A017.
[93] Deshpande, S., and Purwar, A., 2020, "An Image-Based Approach to Variational Path Synthesis of Linkages," J. Comput. Inf. Sci. Eng., **21**(2), p. 021005.
[94] Li, R., Zhang, Y., and Chen, H., 2021, "Learning the Aerodynamic Design of Supercritical Airfoils Through Deep Reinforcement Learning," AIAA J., **59**(10), pp. 1–14.
[95] Dering, M., Cunningham, J., Desai, R., Yukish, M. A., Simpson, T. W., and Tucker, C. S., 2018, "A Physics-Based Virtual Environment for Enhancing the Quality of Deep Generative Designs," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Quebec City, Quebec, Canada, p. V02AT03A015.
[96] Lee, X. Y., Balu, A., Stoecklein, D., Ganapathysubramanian, B., and Sarkar, S., 2019, "A Case Study of Deep Reinforcement Learning for Engineering Design: Application to Microfluidic Devices for Flow Sculpting," ASME J. Mech. Des., **141**(11), p. 111401.
[97] Raina, A., Puentes, L., Cagan, J., and McComb, C., 2021, "Goal-Directed Design Agents: Integrating Visual Imitation With One-Step Lookahead Optimization for Generative Design," ASME J. Mech. Des., **143**(12), p. 124501.
[98] Lopez, Christian E, Ashour, Omar, and Tucker, Conrad S., 2019, "Reinforcement Learning Content Generation for Virtual Reality Applications," 39th Computers and Information in Engineering Conference of International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Anaheim, CA, p. V001T02A009.
[99] Cunningham, James, Lopez, Christian, Ashour, Omar, and Tucker, Conrad S. , 2020, "Multi-Context Generation in Virtual Reality Environments Using Deep Reinforcement Learning," 40th Computers and Information in Engineering Conference (CIE) of International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Virtual, p. V009T09A072.
[100] Greminger, M., 2020, "Generative Adversarial Networks With Synthetic Training Data for Enforcing Manufacturing Constraints on Topology Optimization," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Virtual, p. V11AT11A005.
[101] Fujita, K., Minowa, K., Nomaguchi, Y., Yamasaki, S., and Yaji, K., 2021, "Design Concept Generation With Variational Deep Embedding Over Comprehensive Optimization," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, IDETC-21, Virtual.
[102] Cang, R., Vipradas, A., and Ren, Y., 2017, "Scalable Microstructure Reconstruction With Multi-Scale Pattern Preservation," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Cleveland, OH, p. V02BT03A010.
[103] Cang, R., Xu, Y., Chen, S., Liu, Y., Jiao, Y., and Yi Ren, M., 2017, "Microstructure Representation and Reconstruction of Heterogeneous Materials Via Deep Belief Network for Computational Material Design," ASME J. Mech. Des., **139**(7), p. 071404.
[104] Fokina, D., Muravleva, E., Ovchinnikov, G., and Oseledets, I., 2020, "Microstructure Synthesis Using Style-Based Generative Adversarial Networks," Phys. Rev. E, **101**(4), p. 043308.
[105] Wang, Z., Xian, W., Baccouche, M. R., Lanzerath, H., Li, Y., and Xu, H., 2021, "A Gaussian Mixture Variational Autoencoder-Based Approach for Designing Phononic Bandgap Metamaterials," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, IDETC-21, Virtual.
[106] Cang, Ruijin, and Ren, Max Yi, 2016, "Deep Network-Based Feature Extraction and Reconstruction of Complex Material Microstructures," 42nd Design Automation Conference of International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Charlotte, NC, p. V02BT03A008.
[107] Vermeer, Kaz, Kuppens, Reiner, and Herder, Justus, 2018, "Kinematic Synthesis Using Reinforcement Learning," 44th Design Automation Conference of International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Quebec City, Quebec, Canada, p. V02AT03A009.
[108] Raina, A., McComb, C., and Cagan, J., 2019, "Learning to Design From Humans: Imitating Human Designers Through Deep Learning," ASME J. Mech. Des., **141**(11), p. 111102.
[109] Puentes, L., Raina, A., Cagan, J., and McComb, C., 2020, "Modeling a Strategic Human Engineering Design Process: Human-Inspired Heuristic Guidance Through Learned Visual Design Agents," Proceedings of the Design Society: DESIGN Conference, Cavtat, Croatia, Cambridge University Press, pp. 355–364.
[110] Yukish, M. A., Stump, G. M., and Miller, S. W., 2020, "Using Recurrent Neural Networks to Model Spatial Grammars for Design Creation," ASME J. Mech. Des., **142**(10), p. 104501.
[111] Zhu, J.-H., Zhang, W.-H., and Xia, L., 2016, "Topology Optimization in Aircraft and Aerospace Structures Design," Arch. Comput. Methods Eng., **23**(4), pp. 595–622.
[112] Xia, L., and Breitkopf, P., 2017, "Recent Advances on Topology Optimization of Multiscale Nonlinear Structures," Arch. Comput. Methods Eng., **24**(2), pp. 227–249.
[113] Borrvall, T., and Petersson, J., 2003, "Topology Optimization of Fluids in Stokes Flow," Int. J. Numer. Methods Fluids, **41**(1), pp. 77–107.
[114] Zhou, S., and Li, Q., 2008, "A Variational Level Set Method for the Topology Optimization of Steady-State Navier–Stokes Flow," J. Comput. Phys., **227**(24), pp. 10178–10195.
[115] Zegard, T., and Paulino, G. H., 2016, "Bridging Topology Optimization and Additive Manufacturing," Struct. Multidiscipl. Optim., **53**(1), pp. 175–192.
[116] Langelaar, M., 2016, "Topology Optimization of 3D Self-Supporting Structures for Additive Manufacturing," Addit. Manuf., **12**, pp. 60–70 .
[117] Dbouk, T., 2017, "A Review About the Engineering Design of Optimal Heat Transfer Systems Using Topology Optimization," Appl. Therm. Eng., **112**, pp. 841–854.
[118] Koga, A. A., Lopes, E. C. C., Nova, H. F. V., De Lima, C. R., and Silva, E. C. N., 2013, "Development of Heat Sink Device by Using Topology Optimization," Int. J. Heat Mass Transfer, **64**, pp. 759–772.
[119] Gatys, L., Ecker, A. S., and Bethge, M., 2015, "Texture Synthesis Using Convolutional Neural Networks," Advances in Neural Information Processing Systems, Montréal, Quebec, Canada.
[120] Gatys, L. A., Ecker, A. S., and Bethge, M., 2016, "Image Style Transfer Using Convolutional Neural Networks," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, pp. 2414–2423.
[121] Berthelot, D., Schumm, T., and Metz, L., 2017, "BEGAN: Boundary Equilibrium Generative Adversarial Networks," arXiv:1703.10717.
[122] Jiang, Z., Zheng, Y., Tan, H., Tang, B., and Zhou, H., 2017, "Variational Deep Embedding: An Unsupervised and Generative Approach to Clustering," Proceedings of the 26th International Joint Conference on Artificial Intelligence, Melbourne, Australia, pp. 1965–1972.
[123] Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., Aitken, A., Tejani, A., Totz, J., and Wang, Z., 2017, "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, IEEE, pp. 105–114.
[124] Isola, P., Zhu, J.-Y., Zhou, T., and Efros, A. A., 2017, "Image-to-Image Translation With Conditional Adversarial Networks," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, pp. 1125–1134.
[125] He, K., Zhang, X., Ren, S., and Sun, J., 2016, "Deep Residual Learning for Image Recognition," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, pp. 770–778.
[126] Hu, J., Shen, L., and Sun, G., 2018, "Squeeze-and-Excitation Networks," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, pp. 7132–7141.
[127] Long, J., Shelhamer, E., and Darrell, T., 2015, "Fully Convolutional Networks for Semantic Segmentation," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Boston, MA, pp. 3431–3440.

aded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2hAAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UiBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb

[128] Bostanabad, R., Zhang, Y., Li, X., Kearney, T., Brinson, L. C., Apley, D. W., Liu, W. K., and Chen, W., 2018, "Computational Microstructure Characterization and Reconstruction: Review of the State-of-the-Art Techniques," Prog. Mater. Sci., **95**, pp. 1–41.
[129] Lee, H., Grosse, R., Ranganath, R., and Ng, A. Y., 2009, "Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations," Proceedings of the 26th Annual International Conference on Machine Learning, Montréal, Quebec, Canada, pp. 609–616.
[130] Yu, S., Zhang, Y., Wang, C., Lee, W.-k., Dong, B., Odom, T. W., Sun, C., and Chen, W., 2017, "Characterization and Design of Functional Quasi-Random Nanostructured Materials Using Spectral Density Function," ASME J. Mech. Des., **139**(7), p. 071401.
[131] Molesky, S., Lin, Z., Piggott, A. Y., Jin, W., Vucković, J., and Rodriguez, A. W., 2018, "Inverse Design in Nanophotonics," Nat. Photonics, **12**(11), pp. 659–670.
[132] Dilokthanakul, N., Mediano, P. A., Garnelo, M., Lee, M. C., Salimbeni, H., Arulkumaran, K., and Shanahan, M., 2016, "Deep Unsupervised Clustering With Gaussian Mixture Variational Autoencoders," arXiv:1611.02648.
[133] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., 2017, "Proximal Policy Optimization Algorithms," arXiv:1707.06347.
[134] Gielis, J., 2003, "A Generic Geometric Transformation that Unifies a Wide Range of Natural and Abstract Shapes," Am. J. Botany, **90**(3), pp. 333–338.
[135] Ha, D., and Eck, D., 2018, "A Neural Representation of Sketch Drawings," International Conference on Learning Representations, Vancouver, British Columbia, Canada.
[136] Kulesza, A., and Taskar, B., 2012, "Determinantal Point Processes for Machine Learning," Found. Trends® Mach. Learning, **5**(2–3), pp. 123–286.
[137] Chen, W., and Ahmed, F., 2021, "MO-PaDGAN: Reparameterizing Engineering Designs for Augmented Multi-Objective Optimization," Appl. Soft. Comput., **113**(A), p. 107909.
[138] Li, J., Xu, K., Chaudhuri, S., Yumer, E., Zhang, H., and Guibas, L., 2017, "Grass: Generative Recursive Autoencoders for Shape Structures," ACM Trans. Graph., **36**(4), pp. 1–14 .
[139] Zou, C., Yumer, E., Yang, J., Ceylan, D., and Hoiem, D., 2017, "3D-PRNN: Generating Shape Primitives With Recurrent Neural Networks," Proceedings of the IEEE International Conference on Computer Vision, Venice, Italy, pp. 900–909.
[140] Groueix, T., Fisher, M., Kim, V. G., Russell, B. C., and Aubry, M., 2018, "A Papier-Mâché Approach to Learning 3D Surface Generation," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, pp. 216–224.
[141] Gao, L., Yang, J., Wu, T., Yuan, Y.-J., Fu, H., Lai, Y.-K., and Zhang, H., 2019, "SDM-NET: Deep Generative Network for Structured Deformable Mesh," ACM Trans. Graph., **38**(6), pp. 1–15 .
[142] Mo, K., Guerrero, P., Yi, L., Su, H., Wonka, P., Mitra, N. J., and Guibas, L. J., 2019, "Structurenet: Hierarchical Graph Networks for 3D Shape Generation," ACM Trans. Graph., **38**(6), pp. 1–19.
[143] Park, J. J., Florence, P., Straub, J., Newcombe, R., and Lovegrove, S., 2019, "DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA.
[144] Liu, S., Giles, L., and Ororbia, A., 2018, "Learning a Hierarchical Latent-Variable Model of 3D Shapes," 2018 International Conference on 3D Vision (3DV), Verona, Italy, IEEE, pp. 542–551.
[145] Qi, C. R., Su, H., Mo, K., and Guibas, L. J., 2017, "Pointnet: Deep Learning on Point Sets for 3D Classification and Segmentation," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, pp. 652–660.
[146] Karnewar, A., and Wang, O., 2020, "MSG-GAN: Multi-Scale Gradients for Generative Adversarial Networks," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Virtual, pp. 7799–7808.
[147] van Hasselt, H., Guez, A., and Silver, D., 2016, "Deep Reinforcement Learning with Double q-Learning," Proceedings of the AAAI Conference on Artificial Intelligence, Phoenix, AZ.
[148] Odena, A., Olah, C., and Shlens, J., 2017, "Conditional Image Synthesis With Auxiliary Classifier GANS," International Conference on Machine Learning, Sydney, Australia, pp. 2642–2651.
[149] McComb, C., Cagan, J., and Kotovsky, K., 2018, "Data on the Design of Truss Structures by Teams of Engineering Students," Data Brief, **18**, pp. 160–163.
[150] Hunter, W., 2017, Topy-Topology Optimization with Python, https://github.com/williamhunter/topy
[151] Iren, D., Ackermann, M., Gorfer, J., Pujar, G., Wesselmecking, S., Krupp, U., and Bromuri, S., 2021, "Aachen-Heerlen Annotated Steel Microstructure Dataset," Sci. Data, **8**(1), pp. 1–9.
[152] Larmuseau, M., Sluydts, M., Theuwissen, K., Duprez, L., Dhaene, T., and Cottenier, S., 2020, "Compact Representations of Microstructure Images Using Triplet Networks," npj Comput. Mater., **6**(1), pp. 1–11.
[153] DeCost, B. L., Hecht, M. D., Francis, T., Webler, B. A., Picard, Y. N., and Holm, E. A., 2017, "UHCSDB: Ultrahigh Carbon Steel Micrograph Database," Integrating Mater. Manufactur. Innovation, **6**(2), pp. 197–205.
[154] Zhao, H., Wang, Y., Lin, A., Hu, B., Yan, R., McCusker, J., Chen, W., McGuinness, D. L., Schadler, L., and Brinson, L. C., 2018, "Nanomine Schema: An Extensible Data Representation for Polymer Nanocomposites," APL Mater., **6**(11), p. 111108.
[155] Chang, A. X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., and Savarese, S., 2015, Shapenet: An Information-Rich 3D Model Repository.
[156] Mo, K., Zhu, S., Chang, A. X., Yi, L., Tripathi, S., Guibas, L. J., and Su, H., 2019, "Partnet: A Large-Scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, pp. 909–918.
[157] Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., and Xiao, J., 2015, "3D Shapenets: A Deep Representation for Volumetric Shapes," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Boston, MA, pp. 1912–1920.
[158] Kim, S., Chi, H.-g., Hu, X., Huang, Q., and Ramani, K., 2020, "A Large-Scale Annotated Mechanical Components Benchmark for Classification and Retrieval Tasks With Deep Neural Networks," Proceedings of 16th European Conference on Computer Vision (ECCV), Virtual.
[159] Willis, K. D., Pu, Y., Luo, J., Chu, H., Du, T., Lambourne, J. G., Solar-Lezama, A., and Matusik, W., 2021, "Fusion 360 Gallery: A Dataset and Environment for Programmatic CAD Construction From Human Design Sequences," ACM Trans. Graph., **40**(4), pp. 1–24.
[160] Nobari, A. H., Rashad, M. F., and Ahmed, F., 2021, "Creativegan: Editing Generative Adversarial Networks for Creative Design Synthesis," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, IDETC-21, Virtual.
[161] Regenwetter, L., Weaver, C., and Ahmed, F., 2022, "Framed: Data-Driven Structural Performance Analysis of Community-Designed Bicycle Frames," arxiv preprint.
[162] Chan, Y.-C., Ahmed, F., Wang, L., and Chen, W., 2021, "Metaset: Exploring Shape and Property Spaces for Data-Driven Metamaterials Design," ASME J. Mech. Des., **143**(3), p. 031707.
[163] Wang, L., van Beek, A., Da, D., Chan, Y.-C., Zhu, P., and Chen, W., 2022, "Data-Driven Multiscale Design of Cellular Composites With Multiclass Microstructures for Natural Frequency Maximization," Composite Structures, **280**.
[164] Jongejan, J., Rowley, H., Kawashima, T., Kim, J., and Fox-Gieg, N., 2016, "The Quick, Draw!-ai Experiment," Mount View, CA, Accessed Feb, **17**(2018), p. 4.
[165] Lopez, C., Miller, S. R., and Tucker, C. S., 2018, "Human Validation of Computer vs. Human Generated Design Sketches," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Quebec, Quebec City, Canada, p. V007T06A015.
[166] Toh, C. A., and Miller, S. R., 2013, "Exploring the Utility of Product Dissection for Early-Phase Idea Generation," International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Portland, OR, p. V005T06A034.
[167] Liang, L., Liu, M., Martin, C., and Sun, W., 2018, "A Deep Learning Approach to Estimate Stress Distribution: A Fast and Accurate Surrogate of Finite-Element Analysis," J. R. Soc. Interface, **15**(138), p. 20170844.
[168] Jiang, Haoliang, Nie, Zhenguo, Yeo, Roselyn, Farimani, Amir Barati, and Kara, Levent Burak, 2020, "StressGAN: A Generative Deep Learning Model for 2D Stress Distribution Prediction," 46th Design Automation Conference (DAC) of International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Virtual, p. V11BT11A023.
[169] Nie, Z., Jiang, H., and Kara, L. B., 2020, "Stress Field Prediction in Cantilevered Structures Using Convolutional Neural Networks," ASME J. Comput. Inf. Sci. Eng., **20**(1), p. 011002.
[170] Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., and Battaglia, P., 2020, "Learning Mesh-Based Simulation With Graph Networks," International Conference on Learning Representations, Virtual.
[171] Kochkov, D., Smith, J. A., Alieva, A., Wang, Q., Brenner, M. P., and Hoyer, S., 2021, "Machine Learning–Accelerated Computational Fluid Dynamics," Proc. Natl. Acad. Sci. USA, **118**(21), pp. 1–8.
[172] Duraisamy, K., Iaccarino, G., and Xiao, H., 2019, "Turbulence Modeling in the Age of Data," Annu. Rev. Fluid Mech., **51**(1), pp. 357–377.
[173] Kim, B., Azevedo, V. C., Thuerey, N., Kim, T., Gross, M., and Solenthaler, B., 2019, "Deep Fluids: A Generative Network for Parameterized Fluid Simulations," Comput. Graph. Forum, **38**(2), pp. 59–70.
[174] Dering, M. L., and Tucker, C. S., 2017, "Generative Adversarial Networks for Increasing the Veracity of Big Data," 2017 IEEE International Conference on Big Data (Big Data), Boston, MA, IEEE, pp. 2595–2602.
[175] Panchal, J. H., Fuge, M., Liu, Y., Missoum, S., and Tucker, C., 2019, "Special Issue: Machine Learning for Engineering Design," ASME J. Mech. Des., **141**(11), p. 110301.
[176] Elgammal, A., Liu, B., Elhoseiny, M., and Mazzone, M., 2017, "CAN: Creative Adversarial Networks Generating "Art" by Learning About Styles and Deviating From Style Norms," 8th International Conference on Computational Creativity, ICCC 2017, Atlanta, GA.
[177] Franceschelli, G., and Musolesi, M., 2021, "Creativity and Machine Learning: A Survey," arxiv preprint.
[178] Chen, M., Radford, A., Child, R., Wu, J., Jun, H., Luan, D., Sutskever, I., and Singh, A., 2020, "Generative Pretraining From Pixels," Proceedings of the 37th International Conference on Machine Learning, Virtual, pp. 1691–1703.
[179] Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., and Sutskever, I., 2021, "Zero-Shot Text-to-Image Generation," International Conference on Machine Learning, Virtual.
[180] Dhariwal, P., Jun, H., Payne, C., Kim, J. W., Radford, A., and Sutskever, I., 2020, "Jukebox: A Generative Model for Music," arxiv preprint.
[181] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I., 2021, "Learning Transferable Visual Models From Natural Language Supervision," International Conference on Machine Learning, Virtual.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/7/071704/6866682/md_144_7_071704.pdf?casa_token=u7fKLIW2kAAAAAA:7DdhAW_cFL2lSY996WGHUjKMW1s7QC1M9UfBmXiDkO-xuw3K-AXEImF11ISL8ntDy_sQpyXO by Universita Di Pisa user on 19 Novemb